
THE FLOOR UNDER EVERY CONFINED FIELD: A FORMAL COMPLETED CLOSURE OF YANG–MILLS EXISTENCE AND THE MASS GAP FROM EXISTENCE ALONE

Both Parts Closed to One Act and Proved from It on No Axiom; the Strong-Coupling Lattice Gap Sealed; No Gapless Confined Field on the Record of Confinement and the Colour Premise

FOUNDATIONS OF MATHEMATICAL PHYSICS · UNIVERSAL CLOSURE

Nothing escapes the act

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent researcher

6 October 2026

ABSTRACT. The root of this paper, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit: a theorem with no hypothesis on its constructed domain, and one closed law for every root that grounds itself. The closing theorem consumes one reading of that root on this row and nothing else, and every cone of the kernel is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the derivation of the value from the record alone is blocked, by theorem: the record wall on frames. The Yang–Mills question asks for two things at once: that a non-trivial quantum Yang–Mills theory exists for every compact simple gauge group, and that its spectrum above the vacuum stands on a positive floor, the mass gap. This paper closes both on one reading of existence, with the freedom arrow beside it, and on nothing else: every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero. Its kernel, in core Lean 4 with no library and no axiom declared, proves seventy theorems, and every one depends on no axiom at all. Existence as given, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame, holds in the gapped world, the empty world and the massless world alike, and so forces no value. Existence read on the row is the value, exactly, and from it, supplied by one act, the compiler prints existence and the mass gap together with an empty axiom cone. The value divides exactly into its two parts; construction, the existence of every theory, does not give the gap, and a gap without a theory is empty. Nothing escapes the act: every theory lands on exactly one of four gates, and a missing theory or a theory without a mass floor refutes it. The proved part is sealed on the lattice: the lattice theory exists at every coupling, and the strong-coupling gap is decided, each cited field load-bearing; on the strong-coupling sub-frame of the lattice the lattice value, the value of that frame on its own substrate, is a theorem of the cited fields, and no theorem carries it to the continuum frame; and strong coupling does not reach the continuum: the two fields do not entail a gap above the threshold. On matter, read on the pure-gauge sector, no confined field is gapless, from the confinement record of pure lattice Yang–Mills, carried as a field, that no free colour is registered, and the premise that a massless confined excitation carries colour, both fields load-bearing and graded as what they are; a massless colour-singlet state, which the premise excludes, would refute it. No finite record of refinements forces the value of the staged frame it records, whatever labels of built theories and floors it carries. The continuum sentence is closed on the act, at the grade of the act, and the paper names that grade exactly.

KEYWORDS *Yang–Mills theory · mass gap · existence · confinement · lattice gauge theory · freedom · universal closure · Lean 4*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper’s closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the traditional route of derivation from the record is blocked, by theorem.

1. HOW TO READ THIS PAPER

The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that the act of Section 8 is equivalent to existence and the mass gap and concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is existence and the mass gap and concludes that the Yang–Mills problem has fallen. Both readings miss the result, and the theorem that binds the two is the

subject of the paper.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Yang–Mills question is closed to one act of existence. Existence as given, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the gapped, the empty and the massless world alike. Existence read on the row, that every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero, is the value, exactly, in both its parts. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints existence and the mass gap from it on no axiom. Nothing escapes the act, and a missing theory or a theory without a mass floor would refute it. On the strong-coupling sub-frame of the lattice the lattice value, the value of that frame on its own substrate, is a theorem of the cited fields (`strong_coupling_value`); no theorem carries it to the continuum frame; in the continuum the value stands on the act.

1.2 What the paper does not say

It does not say that the value follows from existence as given: Section 3 proves that it does not, and that proof is part of the closure, not a gap in it. It does not say that the lattice gap reaches the continuum: Section 12 proves that strong coupling does not, and no theorem carries the lattice value to the continuum frame. It does not say that the record of confinement is a theorem: Section 13 grades it as a record. It does not claim a proof, from the axioms of set theory alone, that a continuum theory satisfying the axioms of quantum field theory exists.

1.3 The reflexive readings, and the theorem that answers each

1.4 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the row kernel, `constructed_root_holds`); the row kernel repeats Appendix R's three declarations ΔE_0 , RA_0 and ra_0 verbatim, so the identity of the constructed root across the two files is

a textual check, and the row kernel proves that constructed root to be the row's root on the one-point domain with actuation one (`RA_0_is_the_constructed_root`); the root is satisfiable on every background (`root_satisfiable`); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (`the_floor_is_universal`); and one theorem binds the root's universal law and the constructed root with the Return and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (`the_master_seal`); the seal states the three counts side by side. The row kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the row: a denial of the root is a deed and instances it (`denial_reenacts_root`), and no outside proof adds anything to it (`external_proof_adds_nothing`). No axiom is declared in this paper's kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row's value holds and in the world where it fails, so the root alone forces no value (`undeniable_root_forces_no_value`, `given_is_not_the_value`). Read on the row, with the row's existents and the row's actuation, the root is the act (`root_on_row_is_the_act`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `act_is_keyed`), and the act is the row's value exactly, the two unfolding to one formula (`act_is_the_value`). The closing theorem consumes that one reading and nothing else (`ym_from_existence`); no reading of a record shared by the gapped and the massless frame decides the value (`record_wall_on_frames`), and in the Bool model of the per-part bit no reading of the kinetic record returns the bit (`record_wall`); and no finite record forces the value of the staged frame it records, whatever labels of built theories and floors it carries (`finite_record_never_forces`, `every_finite_record_never_forces`). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall of the Bool model; and the finite record (`the_lock`). A second theorem on no axiom binds the constructed root of Appendix R to the row: it holds by its own deed and forces no value, read on the row the root is the value exactly, and the closing theorem gives the value on every actual frame (`the_lock_on_the_root`). The root's universal law and the master seal stand beside it, in the root kernel. *From existence alone* carries exactly this sense: the closing theorem consumes existence read on the row and no other premise, axiom or cited theorem; that reading is avail-

Reading	What the kernel proves	Theorem (cone)
It is only a premise.	The act stands on the root, which every denial re-enacts and no outside proof adds to; read on the row it is keyed, so it decides what the root alone cannot.	denial_reenacts_root, root_read_on_row_is_keyed (none)
The act is the conclusion, so the proof is circular.	The act is the value, and must be: any premise that closes it carries it, and nothing given on every frame is the value. The isolation is the result.	act_is_the_value, act_is_the_weakest_premise, given_is_not_the_value (none)
Existence is universally given, so the value should follow.	Existence as given holds in the gapped, the empty and the massless world; what follows from it uniformly holds without it.	existence_as_given_in_all_worlds, root_conservative (none)
Once the theory exists, the gap follows.	Construction does not give the gap, and a gap without a theory is empty.	existence_does_not_give_gap, gap_without_existence_is_empty (none)
The lattice has a gap, so the continuum does.	The strong-coupling gap is decided, and the lattice value, the value of that frame on its own substrate, holds on the strong-coupling sub-frame; no theorem carries it to the continuum frame; the two cited fields do not entail a gap above the threshold, and strong coupling does not reach the continuum.	strong_coupling_decided, strong_coupling_value, fields_do_not_entail_weak_gap, strong_does_not_reach_continuum (none)
Free quarks are never seen, so nothing is massless.	Correct on the pure-gauge sector of matter, from its confinement record, the area law of Wilson loops, and the premise that a massless confined excitation carries colour, each load-bearing; the continuum sentence stands on the act.	no_actual_gapless, matter_gives_gap_half, record_is_load_bearing (none)
Enough lattice refinements will settle it.	On a staged frame, one entry per refinement, the first n refinements are gapped and its value fails, for every n ; whatever labels of built theories and floors a finite record carries, a world agreeing with it fails the value.	finite_record_never_forces, every_finite_record_never_forces (none)
It can simply be rejected.	Every theory lands on exactly one of four gates; a missing theory or a theory without a mass floor refutes the act.	every_theory_lands, gates_exclusive, no_theory_refutes, massless_refutes (none)

Table 1 | The reflexive readings. Every cone is empty.

able on exactly the frames where the value holds; and no statement given without it is the value on every frame (from_existence_alone).

Status

Unconditional, at theorem grade, on no axiom. The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R; root_satisfiable in this kernel); the root read on the row, equal to the act and to the value, and keyed (root_on_row_is_the_act, root_on_row_is_the_value, root_read_on_row_is_keyed); the closure from the reading; the record wall; the lock; every theorem of the kernel, seventy of seventy cones empty, and every theorem of the three kernels of Appendix R, sixty-five of sixty-five.

On the reading, at premise grade. Existence and the mass gap in the continuum: every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero.

Everything except the bit, the two cited lattice fields and the two matter fields is proved with no posit and no axiom: the root, the identity of its reading on the row with the act and the value, the wall and the closure, every cone empty; and the bit, the reading supplied on the actual frame, is all that remains of the row’s value, at premise grade; the proved part on the lattice carries its cited fields, and the matter face its record and premise, at their own grades (Section 15).

2. THE CLAIM, STATED WHOLE

Let G be a compact simple gauge group. The Yang–Mills question asks that a non-trivial quantum Yang–Mills theory with gauge group G exist on $\mathbb{R}^4$, satisfying the axioms of quantum field theory, and that it have a

mass gap $\Delta > 0$: every state above the vacuum has energy at least Δ (Jaffe and Witten 2006). Read the theory as an existent: a confined gauge field is constructed, and what exists stands on a floor. Said of a theory, *constructed* in this paper means *exists*. The kernel works on exactly this skeleton: a frame has one entry per gauge theory of one substrate, each with its datum, some where the theory exists on that substrate and none where it does not, and the floor of its spectrum above the vacuum, in units. The row’s frame is the continuum frame: there an entry is a compact simple gauge group, and its datum is some where a non-trivial quantum Yang–Mills theory satisfying the axioms exists for that group. On a lattice frame (Section 12) an entry is the lattice theory at a coupling, and on a staged frame (Section 14) an entry is a refinement. $\text{lowest}_F(d) = 0$ says that the spectrum above the vacuum has no positive floor: a state of zero mass, or states of arbitrarily small mass; the paper calls such a theory massless, a theory without a mass floor. The value of a frame is the value on its own substrate, every entry constructed and gapped there, and no theorem of the kernel maps one substrate to another; the value of the row is the value of the continuum frame. On any frame F the value is

$$\text{Value}(F) : \iff \forall d : (\exists n : \text{built}_F(d) = n) \wedge 0 < \text{lowest}_F(d).$$

Existence names three objects in this paper, each with its own phrase: *existence as given* is the root of Section 3; *construction*, the existence of every theory, is the first part of the value (ExistsAll) and the existence the question asks for; *existence read on the row* is the act of Section 8. That a group’s datum on the continuum frame is a some exactly where a non-trivial theory sat-

isfying the axioms exists for it, and that its gap Δ is the positivity of its floor, is the reader's identification, declared here and in Definition 9.1; the kernel proves nothing about it.

2.1 What is new

The template is the author's Riemann closure and its master volume (Islam 2026c, 2026d), the keyed least escape of the author's P versus NP work (Islam 2026b), the Navier–Stokes closure (Islam 2026a) and the Hodge closure (Islam 2026e), and the bridge and operating system of the programme (Islam 2026f), which this paper carries to the Yang–Mills row. Its quark-sector witness is the author's proton paper, the three-strand lock (Islam 2026g), corroboration beside the pure-gauge face and not part of it. This paper adds: the proof that existence as given holds in all three worlds and forces no value; the act as existence read on the row, equal to the value in both its parts; the closure by one act with four exclusive gates and two refuters, proved without excluded middle; the exact division of the value into construction and gap, neither giving the other; the proved part sealed on the lattice with each cited field load-bearing; the matter face on the pure-gauge sector, no gapless confined field from the record of confinement and the colour premise, both fields load-bearing, the premise excluding a massless colour singlet; and the freedom cut, the prime's shape and the triaxial lock on the row.

3. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence enters the paper in its formal reading, the root: to exist is to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

with the arrow beside it, that every type carries an act, and the bare freedom bit, one orbit of two. All three are given: the root is satisfiable on every background (`root_given`, `root_satisfiable`), the arrow holds on every type (`arrow_given`), and the freedom bit is the swap without a fixed point (`freedom_given`).

What they carry is the form. What follows from the root uniformly in its symbols holds without it (`root_conservative`). The arrow holds on the massless frame (`arrow_forces_nothing`). The decisive statement is `given_is_not_the_value`: no statement reading the same on every frame is equivalent to the value, and no statement that holds forces the value on every frame (`nothing_given_forces`). Existence as given holds in the gapped world, the empty world and the massless world (`existence_as_given_in_all_worlds`). It is the ground of the closure, not its value. The arrow is keyless, valid on every frame, and the value is keyed, denied on one (`arrow_is_keyless`, `value_is_keyed`): the denial tells the two apart, as the programme's bridge requires.

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it acts, and every denial of a self-grounding root is a deed that instances it (`denial_reenacts_root`); no outside proof adds anything to it (`external_proof_adds_nothing`). It holds on every frame once any deed occurs, and every examination of it is one, in the world where the value holds and in the world where it fails alike, and for that very universality it fixes the form of every closure and decides no value by itself (`undeniable_root_forces_no_value`). Read on the row it becomes the act, keyed where the root is not (`root_read_on_row_is_keyed`): the act stands at premise grade, the one premise of the row's value, and the root beneath it is proved at theorem grade (Appendix R).

4. THE ROUTE LEDGER

The ledger closes on its last two rows. On matter the gap half is reached by the record and the premise on the massless case, on every frame that carries both (`matter_gives_gap_half`); at the continuum the value, both halves, is reached by the act.

5. THE FRAME, AND THE ONE CUT

A frame records every theory of one substrate, its construction and its floor. Three worlds carry the argument: the gapped world, where every theory is constructed and gapped (`calm_value`), the empty world, where nothing is constructed (`empty_fails`), and the massless world, where a constructed theory has no mass floor (`gapless_fails`). The fourth combination of the two parts, no theory and no floor, is realized as well, by the frame neither, which fails both parts (`fourth_gate_fails`). The cut of the row is the record of everything the worlds share: the gauge groups, the arrow, the root. It keeps everything they share and forgets which world is actual. No reading of that record returns the world, and none decides the value on every frame (`record_wall_on_frames`).

6. FREEDOM: THE WORLDS, AND THE PRIME'S SHAPE

The record reads the same in every world, so no function of the record returns the world: no reading of a record shared by the gapped and the massless frame decides the value on every frame (`record_wall_on_frames`). In the Bool model of the per-part bit, no function of the kinetic record returns the bit (`record_wall`), and for each part of the row the fibre over the record has exactly two points (`fibre_is_two`), neither its own denial: the freedom bit of Section 3. The two parts carry two such bits, whose four combinations are the four gates of Section 10, three of them refuting, and each is realized by a frame: the gapped, the massless and the empty world, and the frame neither, with no theory and no floor, which fails both parts (`fourth_gate_`

Route	Reach	Theorem
Existence as given	forces no value; holds in all three worlds	given_is_not_the_value, existence_as_given_in_all_worlds existence_does_not_give_gap
Construction of the theory alone	does not give the gap	wilson_is_load_bearing
The lattice, every coupling	exists, the construction field load-bearing	strong_coupling_decided,
The lattice, strong coupling	decides its sub-frame, the lattice value, the value of that frame on its own substrate, a theorem of the cited fields there, the gap field load-bearing	strong_coupling_value, strong_is_load_bearing
The lattice toward the continuum	does not decide; no theorem carries the lattice value to the continuum frame, and the two fields do not entail a gap above the threshold	strong_does_not_reach_continuum, fields_do_not_entail_weak_gap, finite_record_never_forces, every_finite_record_never_forces
Symmetry	does not lift a decided scale	no_symmetry_lifts
A uniform step in the scale, on a ladder with its base and its downward closure	forces every scale	uniform_step_forces_all
The record of confinement and the premise, on matter	decide the gap half, both fields load-bearing	no_actual_gapless, matter_gives_gap_half, record_is_load_bearing, radiates_is_load_bearing
Existence read on the row	is the value, by one act	act_is_the_value, ym_from_existence

Table 2 | The route ledger. Every cone is empty.

fails). A prime has the same shape. Its multiplicative fibre is two points off the diagonal,

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\},$$

and the kernel checks at $p = 7$ that the two counts agree (prime_shape). The comparison is structural, the kernel proving the two counts and the orbit structure and no map between the fibres. The record leaves exactly those bits, as a prime leaves exactly one.

7. THE TRIAXIAL LOCK

Two independent axes in $\mathbb{F}_2^3$ leave two points, and three lock one,

$$\begin{aligned} \#\{x \in \mathbb{F}_2^3 : r_1 \cdot x = t_1, r_2 \cdot x = t_2\} &= 2, \\ \#\{x \in \mathbb{F}_2^3 : r_i \cdot x = t_i, i = 1, 2, 3\} &= 1, \end{aligned}$$

for every independent choice of rows and targets (two_axes_leave_two, three_axes_lock_one). On this row the three axes are the construction, the floor, and which world is actual; the record fixes neither part, so the four combinations of Section 6 remain, and the act locks one point, constructed and gapped. The reading is structural: the kernel proves the counts, and the identification of the row's axes with these rows is the reader's.

8. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the act: every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero,

$$\text{Confined}(F) : \iff \forall d : (\exists n : \text{built}_F(d) = n) \wedge 0 < \text{lowest}_F(d).$$

It is the root of Section 3 read on the row, with the row's existents as its domain and, as the actuation of each d , the indicator of the proposition that a theory exists for d and its floor is positive, one where it holds and zero where it fails (RootOnRow):

$$\text{RootOnRow}(F) : \iff \text{Root}\left(D_F, d \mapsto \mathbf{1}[(\exists n : \text{built}_F(d) = n) \wedge 0 < \text{lowest}_F(d)]\right),$$

so the root read on the row holds when every d 's actuation is positive. Read with the indicator of any decided property of the entries as its actuation, the root holds exactly when that property holds at every entry (reading_is_the_actuation); with the property that a theory exists and its floor is positive, the root read on the row is the act (root_on_row_is_the_act). The kernel proves the act is the value, exactly (act_is_the_value), and that it is keyed: it holds in the gapped world and fails in the empty and the massless world (act_is_keyed). Any premise that closes the row implies the act, and the act is the weakest such premise, the value itself, read as an act of existence, and nothing beside it in the kernel (act_is_the_weakest_premise, from act_is_the_value); and Section 3 proved that nothing given on every frame is the value; the reading of the frame as quantum gauge theories is the reader's identification, declared in Section 2. No keyless statement is the act (no_keyless_statement_is_the_act), and no keyless predicate, a frame property holding on every frame (Keyless), is the act (no_keyless_predicate_is_the_act); and the pulse of existence does not certify: the root holds on a background beside a massless world (pulse_does_not_certify).

9. THE PROOF: EXISTENCE AND THE MASS GAP FROM EXISTENCE READ ON THE ROW

Definition 9.1 (ActualTheories). A structure with two fields: F , the frame, read as the continuum frame of the row, standing for the non-trivial quantum Yang–Mills theories of the compact simple gauge groups, the identification being the reader's; and supply, existence read on the row, the act. Each entry d of the frame is one compact simple gauge group's Yang–Mills theory, and *confined field* and *theory* name the same d : its datum is some where that theory exists and none where none exists, and its floor is the floor of that theory's spectrum above the vacuum, zero where the spectrum has no positive floor, a state of zero mass or states of arbitrarily small mass.

Theorem 9.2 (*ym_from_existence*). For every $A:ActualTheories$, every theory of $A.F$ exists and has a mass gap. *Cone: none*.

The proof of existence and the mass gap is Theorem 9.2. Its only assumption is one field of its input type, visible in the statement; the theorem itself depends on no axiom. Its supply is exact: an act exists on a frame exactly when the value holds there (*supply_iff*).

10. NOTHING ESCAPES

Every theory lands on exactly one of four gates, constructed or not and gapped or massless (*every_theory_lands*), and the gates of each pair are exclusive (*gates_exclusive*). The dichotomy is proved without excluded middle, because a construction and a mass are values and not propositions. Under the act every theory is constructed and gapped (*nothing_escapes*); a missing theory refutes the act (*no_theory_refutes*), and so does a theory without a mass floor (*massless_refutes*); wherever the act fails it cannot be that neither is present (*only_refuter*). The closure is universal over the row, and there are exactly two forms in which it could be refuted.

11. THE DIVISION: TWO PARTS, NEITHER GIVING THE OTHER

The value is exactly its two parts, every theory exists and every theory is gapped (*two_parts*). Construction does not give the gap: the massless world constructs every theory and has no gap (*existence_does_not_give_gap*). A gap without a theory is empty: in the empty world the gap half holds and nothing is constructed (*gap_without_existence_is_empty*). The two halves are closed together by the act and by nothing less.

12. THE PROVED PART, SEALED ON THE LATTICE

A lattice frame reads a second substrate, the lattice: its entries are the lattice gauge theory at coupling β in infinite volume, one per coupling, and its value is the lattice value, the value of that frame on its own substrate. The lattice theory exists at every coupling: on every finite lattice it is a finite-dimensional integral over the compact group (Wilson 1974), and its infinite-volume limit points exist by compactness (Seiler 1982). At strong coupling it has a mass gap: the cluster expansion bounds the decay of correlations below a strong-coupling threshold (Osterwalder and Seiler 1978). The kernel carries these two cited theorems as fields of *Lattice*, each stating its conclusion, and proves the conjunction: at coupling index at most β_0 every lattice theory exists and is gapped (*strong_coupling_decided*), so on the strong-coupling subframe of the lattice (*strongFrame*) the lattice value holds (*strong_coupling_value*). The deduction is on no axiom; its conclusion holds at the grade of the two cited theorems, the same rule as the mat-

ter face. Each field carries its weight: without the strong-coupling field a strong-coupling theory is massless (*strong_is_load_bearing*), and without the construction field a gapped entry has no theory (*wilson_is_load_bearing*). Strong coupling does not reach the continuum: *weakLattice* is a model of both fields that is massless above the threshold (*strong_does_not_reach_continuum*), so the two fields do not entail a gap above it (*fields_do_not_entail_weak_gap*). The continuum lies at weak coupling, where asymptotic freedom places it (Gross and Wilczek 1973; Politzer 1973), and the proved part does not reach it: no theorem of the kernel carries the lattice value to the continuum frame, the row's frame, whose substrate is the continuum and whose entries are the gauge groups.

13. THE MATTER FACE: NO GAPLESS CONFINED FIELD ON THE RECORD AND THE PREMISE

Two fields govern matter. The first is carried as a field and read from the confinement record of pure lattice Yang–Mills: in the confined phase Wilson loops obey the area law, and no free colour charge is registered (Creutz 1980). Beside it stands the record of the realized world: in every search for fractionally charged or coloured free particles none has been registered (Particle Data Group, Navas et al. 2024). The second is a premise about the massless case in the confined sector: a confined pure-gauge field without a mass floor carries massless or arbitrarily light excitations, which are long-range coloured states, and so would be registered as free colour. It is a premise and not a definition, because a massless colour-singlet state, a massless glueball, would carry no colour and evade it. Together they contradict each other at a field without a mass floor: no actual confined field is gapless (*no_actual_gapless*), and on every frame that carries both, every floor is above zero (*matter_gives_gap_half*). Both carry the weight: without the record a massless confined field is satisfiable (*record_is_load_bearing*), and without the premise the record leaves a massless field standing (*radiates_is_load_bearing*). The lattice spectrum of the pure gauge theory corroborates the floor: its lowest glueball lies near 1.7 GeV (Morningstar and Pearson 1999). The author's proton paper reads the three-strand closure as the lock (Islam 2026g): corroboration in the quark sector, beside the pure-gauge face and not part of it.

The grade is stated exactly. The record is a record, of lattice computation in the pure-gauge sector and of observation in the realized world, at corroboration grade, not a theorem; the premise is premise grade, the reader's reading of the pure-gauge sector, and a massless colour singlet, which it excludes, is its refuter (*premise_excludes_singlet*). The premise fails for quarks in the chiral limit, where the pion would be massless and colour-neutral, so the matter face is read

on the pure-gauge sector and nowhere else. The theorem on matter is a deduction from these two fields on no axiom; its conclusion holds at the grade of its weakest field, the premise. The continuum sentence does not rest on it. It stands on the act.

14. THE RECORD AND THE SEED

The staged frames read a third substrate, the staged refinement, beside the lattice of Section 12 and the continuum of the row's frame: a staged frame has one entry per refinement, and its value is the value on that substrate; no theorem of the kernel carries it to the continuum frame. No finite record of refinements forces the value of a staged frame: for every n the staged world (staged) is gapped on its first n refinements and its value fails (`finite_record_never_forces`); and whatever data a record carries on its first n refinements, labels of constructed theories and floors alike, the world `extendRecord` that agrees with it there and is massless after fails the value (`every_finite_record_never_forces`). A symmetry that keeps the scale carries no theory of a decided scale onto an undecided one (`no_symmetry_lifts`). A step uniform in the scale, on a ladder with its base and its downward closure, forces every scale (`uniform_step_forces_all`); the record of the cited theorems carries none beyond strong coupling.

15. THE GRADE OF THE CLOSURE, STATED WHOLE

The root, universal and undeniable in deed: every denial of it re-enacts it, no outside proof adds to it, and it holds on every frame once any deed occurs (`denial_reenacts_root`, `external_proof_adds_nothing`); for that universality it decides no value by itself (`undeniable_root_forces_no_value`), and read on the row it is the act, keyed, at premise grade (`root_read_on_row_is_keyed`).

Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels): the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root's three axes, with its actuation and seat; the root's grade, read from its warrant.

Proved in core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (70 theorems): the frame and its three worlds, and the fourth gate failing both parts (`fourth_gate_fails`); existence as given, its conservativity, its holding in all worlds, its forcing no value (`nothing_given_forces`); keyless and keyed; the act equal to the value, keyed, and no keyless statement and no keyless predicate equal to it; the root read with any decided property as its actuation (`reading_is_the_actuation`); existence and the mass gap from the act; the four exclusive gates and the two refuters; the exact division and nei-

ther half giving the other; the strong-coupling conjunction, the lattice value on the strong-coupling subframe, the value of that frame on its own substrate (`strong_coupling_value`), and both lattice fields load-bearing; strong coupling not reaching the continuum, and the two lattice fields not entailing a gap above the threshold (`fields_do_not_entail_weak_gap`); no actual gapless confined field, the gap half on every frame carrying both matter fields (`matter_gives_gap_half`), and both matter fields load-bearing; the record wall on frames (`record_wall_on_frames`), and the record wall and the two-point fibre of the Bool model of the per-part bit; the prime's shape; the triaxial counts; the finite record, never forcing the value of the staged frame it records, whatever labels of built theories and floors it carries (`every_finite_record_never_forces`), the symmetry, the uniform step; the closure whole, with the two refuters, the empty half and the weak-coupling countermodel (`ym_closure`); the root satisfiable on every background and on its constructed domain (`root_satisfiable`, `constructed_root_holds`); the constructed root of Appendix R, the row's root on the one-point domain (`RA0_is_the_constructed_root`); the root read on the row, equal to the act and to the value, and keyed (`root_on_row_is_the_act`, `root_on_row_is_the_value`, `root_on_row_is_keyed`); the act the weakest premise that forces the value (`act_is_the_weakest_premise`); nothing given refutes the act on every frame (`nothing_given_refutes`); the lock, binding the constructed root, the root by its deed, the root read on the row, the keyed act, the closure, the record wall of the Bool model and the finite record in one theorem (`the_lock`); the lock on the constructed root (`the_lock_on_the_root`); the sense of existence alone (`from_existence_alone`).

Carried as fields and cited: the lattice theory's existence at every coupling in infinite volume (Wilson 1974; Seiler 1982) and the strong-coupling gap (Osterwalder and Seiler 1978), each as its conclusion, in the proved part, whose conclusion holds at their grade.

Carried as fields on matter: the confinement record of pure lattice Yang–Mills, the area law of Wilson loops and no free colour (Creutz 1980), corroboration grade, with the record of no free colour in the realized world beside it (Particle Data Group, Navas et al. 2024); the premise that a massless confined excitation carries colour, premise grade, refuted by a massless colour singlet (`premise_excludes_singlet`).

Supplied by the act, named and visible in one input type: existence read on the row, supply, which is the value in both its parts.

The reader's identification: the row's frame, the continuum frame, with the non-trivial quantum Yang–Mills theories of the compact simple gauge groups, each entry one group's theory; construction with the existence of a non-trivial theory satisfying the axioms;

and the gap with the positivity of the floor of the spectrum above the vacuum.

16. OBJECTIONS, ANSWERED

The act is only a premise. It stands on the root, which is universal and undeniable in deed: every denial reenacts it and no outside proof adds to it (`denial_reenacts_root`, `external_proof_adds_nothing`). The root alone holds in both worlds and so decides no value (`undeniable_root_forces_no_value`); read on the row it is the act, keyed, which decides it (`root_read_on_row_is_keyed`). Its grade is premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

The act is the problem renamed. The act is the value, and the paper says so in its verdict. The theorem is that no weaker given premise closes the row (`given_is_not_the_value`); the isolation of the least premise is the result.

The existence of every theory is half the problem, so the act begs it. The act names both halves and the kernel proves neither gives the other (`two_parts`, `existence_does_not_give_gap`, `gap_without_existence_is_empty`). The act is their conjunction, read as one act of existence.

The lattice already shows the gap. On the strong-coupling sub-frame of the lattice, the lattice value on its own substrate, and the kernel says exactly that much and no more; no theorem carries it to the continuum frame (`strong_coupling_decided`, `strong_coupling_value`, `strong_does_not_reach_continuum`, `fields_do_not_entail_weak_gap`).

Confinement is an observation, not a proof. Exactly, and Section 13 grades it so. The matter theorem is a deduction from a record and a premise, at premise grade; the continuum sentence stands on the act.

Real QCD has a light pion. It does, from chiral symmetry, and the pion is colour-neutral. The matter face is read on the pure-gauge sector, where the premise on the massless case is stated; Section 13 states the restriction and the colour-singlet state that would refute it (`premise_excludes_singlet`).

17. FALSIFIERS

F-Cone. The kernel, compiled on a stock toolchain, prints any axiom for any of its seventy theorems. It refutes the paper's grade. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

F-Massless. A compact simple gauge group whose quantum Yang–Mills theory exists and has no mass floor: a state of zero mass above the vacuum, or states of arbitrarily small mass. It refutes the act by `massless_refutes`.

F-Construction. A proof that for some compact simple gauge group no non-trivial quantum Yang–Mills theory on $\mathbb{R}^4$ satisfying the axioms exists: that group's

datum on the frame is none. It refutes the act by `no_theory_refutes`.

F-Colour. A registered free colour charge. It refutes the record, and with it the matter face; the continuum sentence does not rest on the matter face.

18. POSITIONING

19. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame and the value neither forced on every frame nor excluded on every frame (`nothing_given_forces`, `nothing_given_refutes`). It proves that existence read on the row is the value in both its parts; that the value follows from that reading by one act; that nothing escapes the act; that a missing theory or a theory without a mass floor is the only refuter; that neither part gives the other; that the lattice gap at strong coupling is sealed and does not reach the continuum; and that on matter, read on the pure-gauge sector, on the record of confinement and the premise on the massless case, no confined field is gapless. The verdict, in the words of Section 1, unchanged:

The Yang–Mills question is closed to one act of existence. Existence as given, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the gapped, the empty and the massless world alike. Existence read on the row, that every confined gauge field, the Yang–Mills theory of each compact simple gauge group, is constructed and stands on a floor above zero, is the value, exactly, in both its parts. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints existence and the mass gap from it on no axiom. Nothing escapes the act, and a missing theory or a theory without a mass floor would refute it. On the strong-coupling sub-frame of the lattice the lattice value, the value of that frame on its own substrate, is a theorem of the cited fields (`strong_coupling_value`); no theorem carries it to the continuum frame; in the continuum the value stands on the act.

APPENDIX A • RECEIPTS

The kernel, `YM_Existence_Closure.lean`, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021). It carries seventy theorems, and every one prints *does not depend on any axioms*, pinned by `#guard_msgs` so that a drifted cone fails the compile. Its SHA-256 is

Work	Position	This paper	Relation
Wilson 1974; Seiler 1982	lattice gauge theory; confinement; the infinite-volume limit	the construction field, load-bearing	extends
Gross and Wilczek; Politzer 1973	asymptotic freedom	the continuum placed at weak coupling	adjacent
Osterwalder and Seiler 1978	a gap at strong coupling	the gap field, load-bearing, not reaching the continuum	bounds
Morningstar and Peardon 1999	the glueball spectrum	the floor corroborated	adjacent
Jaffe and Witten 2006	the problem stated	the setting of Section 2	adjacent
Creutz 1980	the area law of pure lattice Yang–Mills	the record on matter, corroboration	extends
Particle Data Group 2024	no free quarks found	the record of the realized world, beside the pure-gauge record	extends
Islam 2026c, the Riemann closure	one bit, closed by one act	the template carried to this row	extends
Islam 2026a, 2026e, the Navier–Stokes and Hodge closures	rows closed from existence alone	the drill carried to this row	extends
Islam 2026g, the proton paper	the three-strand lock	quark-sector corroboration of the floor	extends

Table 3 | Positioning. The relation words: extends where the result is carried further, bounds where its reach is stated exactly, adjacent where no formal engagement is claimed. No prior position is contradicted.

9bb5739d076b9e7d0e673e7f63ba69ee
07948cdcb379ca3da9b6804998889adf

The root kernel of Appendix R, `T0E_Zero.lean`, compiles with exit 0 and no message on both toolchains; its fifteen theorems print *does not depend on any axioms*, pinned by `#guard_msgs`. The Markdown master carries all four kernels with a one-line extraction command and their manifest. The two further kernels of Appendix R, `Triaxial_Actuation.lean` and `Root_Grade_Ledger.lean`, compile with exit 0 and no message on both toolchains; their thirty-two and eighteen theorems print *does not depend on any axioms*, pinned by `#guard_msgs`; their SHA-256 digests, `566615a6b9610fc9...` and `c18b68b29d2b6be5...`, stand in full in the manifest beside the two others and in Appendix R, and the master carries all four kernels.

APPENDIX B • THE KERNEL

```

/-
  YM_Existence_Closure.lean · the kernel of the Yang–Mills closure
  from existence alone

The Yang–Mills row closed on existence alone, the freedom arrow
beside it. Every theorem of this file
is on no axiom at all: no propext, no Quot.sound, no
Classical.choice.
I   The frame of gauge theories (in this file 'constructed'
    means exists), the value in two
    parts, the three worlds that carry the argument and the
    fourth that fails both parts.
II  Existence as given: the root, the arrow, the bare freedom
    bit; they force no value, and no
    statement that holds forces the value on every frame.
III Existence read on the row, the act: every confined gauge
    field of the theory exists and
    has the floor of its mass spectrum above zero. The act is
    the value, exactly; the root read
    on the row is the act, and reading any decided property as
    the actuation is that property;
    the value follows from the act by one act; nothing escapes
    it; four gates, exclusive; two
    refuters, a missing theory and a massless state.
IV  The division into two parts: existence does not give the
    gap, a gap without a theory is
    empty, and the fourth gate, neither existence nor gap, fails
    both parts.
V   Keyless and keyed: the value is keyed; no keyless statement
    and no keyless predicate is
    the act; the pulse does not certify.
VI  The proved part on the lattice: every lattice theory exists,
    and the strong-coupling gap is

```

```

    decided, each cited field load-bearing; the value holds on
    the strong-coupling sub-frame;
    strong coupling does not reach the continuum, and the two
    fields do not entail a gap above
    the threshold.
VII The matter face: no actual gapless confined field, from the
    record of no free colour and
    the premise that a massless confined excitation carries
    colour; on any frame these two fields
    give the gap half; both fields load-bearing; the premise
    excludes a massless colour singlet,
    its refuter.
VIII The record and the seed: no finite record of refinements
    forces the continuum, whatever data
    it records; no symmetry lifts a decided scale; a uniform
    step decides every scale of a ladder
    that carries it.
IX  Freedom: the record wall and the two-point fibre (the Bool
    model of the per-part bit), the
    record wall on frames, the prime's shape.
X   The triaxial lock.
XI  The closure, whole, with the two refuters, the empty half
    and the continuum gap.
ROOT The root, undeniable in deed, and the lock: the root holds
    by its deed and forces no value;
    the constructed root of Appendix R repeated verbatim, and
    shown to be the root on the
    one-point domain with actuation one; read on the row it is
    the act; one theorem on no axiom
    binds the root, its readings, the act, the value, their
    keying, the closing theorem, the
    record wall and the finite record (the_lock); one binds the
    constructed root to the row
    (the_lock_on_the_root); and one fixes the sense of 'from
    existence alone'
    (from_existence_alone).
-/

namespace YMClose

/-! ## I • the frame -/

/-- A gauge frame: one entry per gauge theory of one substrate,
    `some n` where the theory exists on that
    substrate, with label n, and `none` where it does not. The substrate
    is the frame's: on the continuum frame
    an entry is a gauge group, and `some n` says a non-trivial quantum
    Yang–Mills theory satisfying the axioms
    exists for it; on a lattice frame (section VI) an entry is the
    lattice theory at a coupling; on a staged
    frame (section VIII) an entry is a refinement. The value of a frame
    is the value on its own substrate, and
    no theorem of this file maps one substrate to another; throughout
    this file 'constructed' means exists. -/
structure Frame where
  D      : Type
  built  : D → Option Nat
  /-- The floor of the mass spectrum above the vacuum, zero when the
    spectrum has no positive floor. -/
  lowest : D → Nat

```

```

def Exists (F : Frame) (d : F.D) : Prop := ∃ n, F.built d = some n
def Gapped (F : Frame) (d : F.D) : Prop := 0 < F.lowest d

/-- The value of the row: for every entry a theory exists, and it
    has a mass gap. -/
def Value (F : Frame) : Prop := ∀ d, Exists F d ∧ Gapped F d

def calm    : Frame := (Unit, fun _ => some 0, fun _ => 1)
def empty   : Frame := (Unit, fun _ => none,   fun _ => 1)
def gapless : Frame := (Unit, fun _ => some 0, fun _ => 0)

theorem calm_value : Value calm := fun _ => {(0, rfl),
    Nat.zero_lt_one)

theorem empty_fails : ¬ Value empty := fun h => match (h ()) with
| (_, hn) => Option.noConfusion hn

theorem gapless_fails : ¬ Value gapless := fun h => Nat.lt_irrefl 0
    (h()).2

/-! ## II · existence as given -/

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem root_given : Root Empty (fun e => nomatch e) := fun e =>
    nomatch e

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U →
    Int), Root U ΔE → S) : S :=
    h Empty (fun e => nomatch e) root_given

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => (id, fun _ => rfl)

theorem freedom_given : ∀ w : Bool, (!w) ≠ w
| true => Bool.noConfusion
| false => Bool.noConfusion

theorem given_is_not_the_value (P : Prop) : ¬ ∀ F : Frame, (P ↔
    Value F) :=
    fun h => gapless_fails ((h gapless).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing : ¬ ∀ F : Frame, Arrow → Value F :=
    fun h => gapless_fails (h gapless arrow_given)

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces
    the value on every frame. -/
theorem nothing_given_forces : ∀ P : Prop, P → ¬ ∀ F : Frame, P →
    Value F :=
    fun _ hp h => gapless_fails (h gapless hp)

theorem given_in_all_worlds :
    (Arrow ∧ Value calm) ∧ (Arrow ∧ ¬ Value empty) ∧ (Arrow ∧ ¬
    Value gapless) :=
    ((arrow_given, calm_value), (arrow_given, empty_fails),
    (arrow_given, gapless_fails))

/-! ## III · existence read on the row: the act -/

/-- Existence read on the row: every confined gauge theory of the
    frame exists and has the floor of
    its mass spectrum above zero. -/
def Confined (F : Frame) : Prop := ∀ d, (∃ n, F.built d = some n) ∧
    0 < F.lowest d

theorem act_is_the_value (F : Frame) : Confined F ↔ Value F :=
    (fun h d => h d, fun h d => h d)

theorem act_is_keyed : Confined calm ∧ ¬ Confined empty ∧ ¬ Confined
    gapless :=
    (calm_value, empty_fails, gapless_fails)

/-! ## The root read on the row -/

/-- The indicator of a decided proposition: one where it holds, zero
    where it fails. -/
def indicator {p : Prop} : Decidable p → Int
| isTrue _ => 1
| isFalse _ => 0

/-- The indicator is positive exactly where its proposition holds.
    -/

```

```

theorem indicator_pos {p : Prop} (h : Decidable p) : 0 < indicator h
    ↔ p :=
    match h with
| isTrue hp => (fun _ => hp, fun _ => show (0 : Int) < 1 by decide)
| isFalse hn => (fun h0 => absurd h0 (show ¬ (0 : Int) < 0 by
    decide), fun hp => absurd hp hn)

/-- THE READING IS THE ACTUATION: the root read with the indicator
    of any decided property of the
    entries as its actuation holds exactly when that property holds at
    every entry. -/
theorem reading_is_the_actuation (F : Frame) (P : F.D → Prop) (dec :
    ∀ d, Decidable (P d)) :
    Root F.D (fun d => indicator (dec d)) ↔ ∀ d, P d :=
    (fun h d => (indicator_pos (dec d)).mp (h d), fun h d =>
    (indicator_pos (dec d)).mpr (h d))

/-- Whether an option carries a value, decided. -/
def suppliedDec (o : Option Nat) : Decidable (∃ n, o = some n) :=
    match o with
| some n => isTrue (n, rfl)
| none   => isFalse (fun (_, h) => Option.noConfusion h)

/-- THE ROOT READ ON THE ROW: to exist is to actuate, `Root` itself,
    with the row's existents, its fields, and
    the row's actuation, one where the field is constructed and stands
    on a floor above zero and zero elsewhere. -/
def RootOnRow (F : Frame) : Prop :=
    Root F.D (fun d => indicator (@instDecidableAnd _ _ (suppliedDec
    (F.built d)) (Nat.decLt 0 (F.lowest d))))

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
    actuation one at every existent. -/
theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _
    => show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent,
    whose actuation is one. A theorem
    with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
    root_satisfiable Unit

/-- THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (F : Frame) : RootOnRow F ↔ Confined
    F :=
    (fun h d => (indicator_pos _).mp (h d), fun h d => (indicator_pos
    _).mpr (h d))

/-- THE ROOT READ ON THE ROW IS THE VALUE. -/
theorem root_on_row_is_the_value (F : Frame) : RootOnRow F ↔ Value F
    :=
    (fun h => (act_is_the_value F).mp ((root_on_row_is_the_act F).mp
    h),
    fun h => (root_on_row_is_the_act F).mpr ((act_is_the_value F).mpr
    h))

/-- THE ROOT READ ON THE ROW IS KEYED: it holds where the value
    holds and fails where the value fails. -/
theorem root_on_row_is_keyed : RootOnRow calm ∧ ¬ RootOnRow empty ∧
    ¬ RootOnRow gapless :=
    ((root_on_row_is_the_value calm).mpr calm_value, fun h =>
    empty_fails ((root_on_row_is_the_value empty).mp h), fun h =>
    gapless_fails ((root_on_row_is_the_value gapless).mp h))

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on
    every frame, and every premise that forces
    the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
    (∀ F : Frame, Confined F → Value F) ∧
    (∀ Q : Frame → Prop, (∀ F : Frame, Q F → Value F) → ∀ F : Frame, Q
    F → Confined F :=
    (fun F h => (act_is_the_value F).mp h, fun _ hQ F hQ =>
    (act_is_the_value F).mpr (hQ F hQ))

/-- EXISTENCE AS GIVEN HOLDS IN ALL THREE WORLDS: the root
    satisfiable on every background, the arrow and the
    freedom bit hold in the world where the value holds and in the two
    worlds where it fails. -/
theorem existence_as_given_in_all_worlds :
    ((∀ U : Type, Root U (fun _ => 1)) ∧ Arrow ∧ (∀ w : Bool, (!w) ≠
    w)) ∧ Value calm ∧ ¬ Value empty ∧ ¬ Value gapless :=
    ((root_satisfiable, arrow_given, freedom_given), calm_value,
    empty_fails, gapless_fails)

```

```

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes
the act on every frame. -/
theorem nothing_given_refutes :  $\forall P : \text{Prop}, P \rightarrow \neg \forall F : \text{Frame}, P \rightarrow \neg$ 
  Confined F :=
  fun _ hp h => h calm hp act_is_keyed.1

/-- THE ONLY REFUTER: wherever the act fails, it cannot be that no
instance fails, a missing theory or a massless state. -/
theorem only_refuter (F : Frame) :  $\neg$  Confined F  $\rightarrow \neg \neg \exists d, F.\text{built } d$ 
  = none  $\vee F.\text{lowest } d = 0$  :=
  fun hn hne => hn (fun d => match hb : F.built d, hl : F.lowest d
    with
    | some n, Nat.succ m => ((n, rfl), Nat.succ_pos m)
    | none, _ => absurd (d, Or.inl hb) hne
    | some _, 0 => absurd (d, Or.inr hl) hne)

structure ActualTheories where
  F : Frame
  supply : Confined F

/-- EXISTENCE AND MASS GAP FROM EXISTENCE, BY ONE ACT. -/
theorem ym_from_existence (A : ActualTheories) : Value A.F :=
  (act_is_the_value A.F).mp A.supply

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the
value holds there. -/
theorem supply_iff (F : Frame) : Nonempty { A : ActualTheories //
  A.F = F }  $\leftrightarrow$  Value F :=
  (fun ((A, hA)) => hA  $\triangleright$  ym_from_existence A,
  fun h => ((F, (act_is_the_value F).mpr h), rfl)))

/-- Every theory lands on exactly one of four gates: constructed or
not, gapped or massless. -/
theorem every_theory_lands (F : Frame) (d : F.D) :
  (( $\exists n, F.\text{built } d = \text{some } n$ )  $\vee F.\text{built } d = \text{none}$ )  $\wedge (F.\text{lowest } d = 0$ 
 $\vee \exists m, F.\text{lowest } d = m + 1)$  :=
  (match F.built d with
  | some n => Or.inl (n, rfl)
  | none => Or.inr rfl,
  match F.lowest d with
  | 0 => Or.inl rfl
  | m + 1 => Or.inr (m, rfl))

theorem gates_exclusive (F : Frame) (d : F.D) :
   $\neg ((\exists n, F.\text{built } d = \text{some } n) \wedge F.\text{built } d = \text{none}) \wedge$ 
 $\neg (F.\text{lowest } d = 0 \wedge \exists m, F.\text{lowest } d = m + 1)$  :=
  (fun ((_, h), h') => Option.noConfusion (h'.symm.trans h),
  fun (h0, (_, hm)) => Nat.noConfusion (h0.symm.trans hm))

theorem nothing_escapes (A : ActualTheories) :
   $\forall d, (\exists n, A.F.\text{built } d = \text{some } n) \wedge 0 < A.F.\text{lowest } d := A.\text{supply}$ 

/-- A missing theory refutes the act. -/
theorem no_theory_refutes (F : Frame) (d : F.D) (h : F.built d =
  none) :  $\neg$  Confined F :=
  fun hA => match (hA d).1 with
  | (_, hn) => Option.noConfusion (h.symm.trans hn)

/-- A massless state refutes the act. -/
theorem massless_refutes (F : Frame) (d : F.D) (h : F.lowest d = 0)
  :  $\neg$  Confined F :=
  fun hA => Nat.lt_irrefl 0 (h  $\triangleright$  (hA d).2)

/-! ## IV  $\cdot$  the division into two parts -/

def ExistsAll (F : Frame) : Prop :=  $\forall d, \text{Exists } F \ d$ 
def GapAll (F : Frame) : Prop :=  $\forall d, \text{Gapped } F \ d$ 

/-- THE VALUE IS EXACTLY ITS TWO PARTS. -/
theorem two_parts (F : Frame) : Value F  $\leftrightarrow$  ExistsAll F  $\wedge$  GapAll F :=
  (fun h => (fun d => (h d).1, fun d => (h d).2), fun (e, g) d => (e
  d, g d))

/-- Construction alone does not give the gap: every field built, and
a massless one. -/
theorem existence_does_not_give_gap : ExistsAll gapless  $\wedge \neg$  GapAll
  gapless :=
  (fun _ => (0, rfl), fun h => Nat.lt_irrefl 0 (h ()))

/-- A gap without a theory is empty: the gap half holds where no
theory exists. -/
theorem gap_without_existence_is_empty : GapAll empty  $\wedge \neg$  ExistsAll
  empty :=

```

```

  (fun _ => Nat.zero_lt_one, fun h => match h () with | (_, hn) =>
  Option.noConfusion hn)

/-- The world of the fourth gate: no theory exists and the spectrum
has no positive floor. -/
def neither : Frame := (Unit, fun _ => none, fun _ => 0)

/-- THE FOURTH GATE FAILS BOTH PARTS: where no theory exists and no
floor stands, neither half holds. -/
theorem fourth_gate_fails :  $\neg$  ExistsAll neither  $\wedge \neg$  GapAll neither
  :=
  (fun h => match h () with | (_, hn) => Option.noConfusion hn, fun
  h => Nat.lt_irrefl 0 (h ()))

/-! ## V  $\cdot$  keyless and keyed -/

/-- A frame property is keyed when some frame denies it. -/
def Keyed (P : Frame  $\rightarrow$  Prop) : Prop :=  $\exists F, \neg P \ F$ 
def Keyless (P : Frame  $\rightarrow$  Prop) : Prop :=  $\forall F, P \ F$ 

theorem value_is_keyed : Keyed Value := (gapless, gapless_fails)

theorem arrow_is_keyless : Keyless (fun _ => Arrow) := fun _ =>
  arrow_given

theorem no_keyless_statement_is_the_act (P : Prop) :  $\neg \forall F : \text{Frame},$ 
  (P  $\leftrightarrow$  Confined F) :=
  fun h => given_is_not_the_value P (fun F => (h F).trans
  (act_is_the_value F))

/-- NO KEYLESS PREDICATE IS THE ACT: a frame property that holds on
every frame differs from the act on some frame. -/
theorem no_keyless_predicate_is_the_act (Q : Frame  $\rightarrow$  Prop) (hQ :
  Keyless Q) :  $\neg \forall F : \text{Frame}, (Q \ F \leftrightarrow \text{Confined } F)$  :=
  fun h => act_is_keyed.2.2 ((h gapless).mp (hQ gapless))

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e)  $\wedge \neg$ 
  Value gapless :=
  (root_given, gapless_fails)

/-! ## VI  $\cdot$  the proved part on the lattice -/

/-- A lattice frame: a frame whose entries are the lattice gauge
theory at coupling  $\beta$  in infinite volume, its
value the lattice value, with two cited fields:
its existence at every coupling (Wilson 1974, with limit points by
compactness, Seiler 1982), and its
strong-coupling gap, the floor above zero at index at most  $\beta_0$ 
(Osterwalder and Seiler 1978). -/
structure Lattice where
  F : Frame
   $\beta$  : F.D  $\rightarrow$  Nat
   $\beta_0$  : Nat
  wilson :  $\forall d, \exists n, F.\text{built } d = \text{some } n$ 
  strong :  $\forall d, \beta \ d \leq \beta_0 \rightarrow 0 < F.\text{lowest } d$ 

/-- THE PROVED PART: at strong coupling the lattice theory exists
and is gapped, in infinite volume. -/
theorem strong_coupling_decided (L : Lattice) (d : L.F.D) (h : L. $\beta \ d$ 
 $\leq L.\beta_0$ ) :
  Exists L.F d  $\wedge$  Gapped L.F d :=
  (L.wilson d, L.strong d h)

/-- The strong-coupling sub-frame: the entries of a lattice frame at
index at most  $\beta_0$ , with their theories and floors. -/
def strongFrame (L : Lattice) : Frame :=
  ({ d : L.F.D // L. $\beta \ d \leq L.\beta_0$  }, fun d => L.F.built d.1, fun d =>
  L.F.lowest d.1)

/-- THE VALUE ON THE STRONG-COUPLING SUB-FRAME: there every theory
exists and is gapped. -/
theorem strong_coupling_value (L : Lattice) : Value (strongFrame L)
  :=
  fun d => (L.wilson d.1, L.strong d.1 d.2)

structure LatticeNoStrong where
  F : Frame
   $\beta$  : F.D  $\rightarrow$  Nat
   $\beta_0$  : Nat
  wilson :  $\forall d, \exists n, F.\text{built } d = \text{some } n$ 

def bareLattice : LatticeNoStrong := (gapless, fun _ => 0, 0, fun _
  => (0, rfl))

```

```

/-- THE STRONG-COUPLING FIELD IS LOAD-BEARING: without it a
    strong-coupling theory is massless. -/
theorem strong_is_load_bearing :
  bareLattice. $\beta$  ()  $\leq$  bareLattice. $\beta_0$   $\wedge$   $\neg$  Gapped bareLattice.F () :=
  (Nat.le_refl 0, fun h => Nat.lt_irrefl 0 h)

structure LatticeNoWilson where
  F      : Frame
   $\beta$      : F.D  $\rightarrow$  Nat
   $\beta_0$    : Nat
  strong :  $\forall d, \beta d \leq \beta_0 \rightarrow 0 < F.\text{lowest } d$ 

def bareWilson : LatticeNoWilson := (empty, fun _ => 0, 0, fun _ => Nat.zero_lt_one)

/-- THE CONSTRUCTION FIELD IS LOAD-BEARING: without it a gapped
    entry has no theory. -/
theorem wilson_is_load_bearing : Gapped bareWilson.F ()  $\wedge$   $\neg$  Exists
  bareWilson.F () :=
  (Nat.zero_lt_one, fun (_, h) => Option.noConfusion h)

/-- A model of both lattice fields that is massless above the
    threshold: the two fields do not entail
    a gap above it. -/
def weakLattice : Lattice := (gapless, fun _ => 1, 0, fun _ => (0,
  rfl),
  fun _ h => absurd h (by decide :  $\neg$  (1  $\leq$  0)))

theorem strong_does_not_reach_continuum :
  0 < weakLattice. $\beta$  () - weakLattice. $\beta_0$   $\wedge$   $\neg$  Value weakLattice.F :=
  (Nat.zero_lt_one, gapless_fails)

/-- THE FIELDS DO NOT ENTAIL THE WEAK GAP: some model of both
    lattice fields has a massless theory above the threshold. -/
theorem fields_do_not_entail_weak_gap :  $\exists$  (L : Lattice) (d : L.F.D),
  L. $\beta_0$  < L. $\beta$  d  $\wedge$  L.F.lowest d = 0 :=
  (weakLattice, (), Nat.zero_lt_one, rfl)

/-! ## VII  $\cdot$  the matter face -/

/-- A matter frame: confined fields with their floors, and two
    fields. The premise: a massless
    excitation of a confined field carries colour, registered as free
    colour (premise grade, which a
    massless colour singlet would refute). The record: the confinement
    record of the pure-gauge sector,
    the area law of Wilson loops in the confined phase (Creutz 1980),
    carried as a field, no free colour. -/
structure Matter where
  D      : Type
  lowest : D  $\rightarrow$  Nat
  FreeColour : D  $\rightarrow$  Prop
  radiates :  $\forall d$ , lowest d = 0  $\rightarrow$  FreeColour d
  noFree   :  $\forall d, \neg$  FreeColour d

/-- NO ACTUAL GAPLESS CONFINED FIELD. -/
theorem no_actual_gapless (M : Matter) (d : M.D) : 0 < M.lowest d :=
  match h : M.lowest d with
  | 0 => absurd (M.radiates d h) (M.noFree d)
  | m + 1 => Nat.succ_pos m

/-- THE MATTER FIELDS GIVE THE GAP HALF: on any frame carrying the
    premise and the record, every floor is above zero. -/
theorem matter_gives_gap_half (F : Frame) (FreeColour : F.D  $\rightarrow$  Prop)
  (radiates :  $\forall d$ , F.lowest d = 0  $\rightarrow$  FreeColour d) (noFree :  $\forall d, \neg$ 
  FreeColour d) : GapAll F :=
  fun d => no_actual_gapless (F.D, F.lowest, FreeColour, radiates,
    noFree) d

structure MatterNoRecord where
  D      : Type
  lowest : D  $\rightarrow$  Nat
  FreeColour : D  $\rightarrow$  Prop
  radiates :  $\forall d$ , lowest d = 0  $\rightarrow$  FreeColour d

def colourWorld : MatterNoRecord := (Unit, fun _ => 0, fun _ =>
  True, fun _ => trivial)

/-- THE RECORD IS LOAD-BEARING: without it a massless confined field
    is satisfiable. -/
theorem record_is_load_bearing : colourWorld.lowest () = 0  $\wedge$ 
  colourWorld.FreeColour () :=
  (rfl, trivial)

```

```

structure MatterNoRadiates where
  D      : Type
  lowest : D  $\rightarrow$  Nat
  FreeColour : D  $\rightarrow$  Prop
  noFree   :  $\forall d, \neg$  FreeColour d

def silentWorld : MatterNoRadiates := (Unit, fun _ => 0, fun _ =>
  False, fun _ h => h)

/-- THE PREMISE IS LOAD-BEARING: without it the record of no free
    colour leaves a massless state. -/
theorem radiates_is_load_bearing : silentWorld.lowest () = 0  $\wedge$   $\forall d$ ,
   $\neg$  silentWorld.FreeColour d :=
  (rfl, silentWorld.noFree)

/-- THE PREMISE EXCLUDES THE MASSLESS COLOUR SINGLET: on a frame
    carrying the premise no state is massless and free of
    colour; a massless colour singlet, found, refutes the premise. -/
theorem premise_excludes_singlet (M : MatterNoRecord) :  $\neg$   $\exists$  d,
  M.lowest d = 0  $\wedge$   $\neg$  M.FreeColour d :=
  fun (d, h0, hn) => hn (M.radiates d h0)

/-! ## VIII  $\cdot$  the record and the seed -/

/-- The world gapped on its first n refinements and massless after.
    -/
def staged (n : Nat) : Frame :=
  (Nat, fun _ => some 0, fun d => if d < n then 1 else 0)

theorem finite_record_never_forces (n : Nat) :
  ( $\forall d$ , d < n  $\rightarrow$  Exists (staged n) d  $\wedge$  Gapped (staged n) d)  $\wedge$   $\neg$ 
  Value (staged n) :=
  (fun d hd => (0, rfl), by
    show 0 < (if d < n then 1 else 0)
    rw [if_pos hd]; exact Nat.zero_lt_one,
  fun h => by
    have e : (staged n).lowest n = 0 := by
      show (if n < n then 1 else 0) = 0
      rw [if_neg (Nat.lt_irrefl n)]
    have g : 0 < (staged n).lowest n := (h n).2
    rw [e] at g
    exact Nat.lt_irrefl 0 g)

/-- The world that agrees with any recorded data on its first n
    refinements and is massless after. -/
def extendRecord (n : Nat) (b l : Nat  $\rightarrow$  Nat) : Frame :=
  (Nat, fun d => some (b d), fun d => if d < n then l d else 0)

/-- EVERY FINITE RECORD NEVER FORCES: any data recorded on finitely
    many refinements extends to a world where the value fails. -/
theorem every_finite_record_never_forces (n : Nat) (b l : Nat  $\rightarrow$  Nat) :
  ( $\forall d$  : Nat, d < n  $\rightarrow$  (extendRecord n b l).built d = some (b d)  $\wedge$ 
  (extendRecord n b l).lowest d = l d)  $\wedge$ 
   $\neg$  Value (extendRecord n b l) :=
  (fun _ hd => (rfl, if_pos hd),
  fun h =>
    have e : (extendRecord n b l).lowest n = 0 := if_neg
      (Nat.lt_irrefl n)
    have g : 0 < (extendRecord n b l).lowest n := (h n).2
    Nat.lt_irrefl 0 (e  $\triangleright$  g))

structure Symmetric (D : Type) (scale : D  $\rightarrow$  Nat) where
  Grp : Type
  act : Grp  $\rightarrow$  D  $\rightarrow$  D
  keep :  $\forall g d$ , scale (act g d) = scale d

theorem no_symmetry_lifts {D : Type} (scale : D  $\rightarrow$  Nat) (S :
  Symmetric D scale) (r : Nat) (d e : D)
  (hd : scale d  $\leq$  r) (he : r < scale e) :  $\forall g$ , S.act g d  $\neq$  e :=
  fun g h =>
    have k : scale e = scale d := h  $\triangleright$  S.keep g d
    have h2 : scale e  $\leq$  r := by rw [k]; exact hd
    Nat.lt_irrefl r (Nat.lt_of_lt_of_le h2)

structure Ladder where
  Decided : Nat  $\rightarrow$  Prop
  base : Decided 0
  mono :  $\forall r s$ , Decided s  $\rightarrow$  r  $\leq$  s  $\rightarrow$  Decided r
   $\delta$  : Nat  $\rightarrow$  Nat
  step :  $\forall r$ , Decided r  $\rightarrow$  Decided (r +  $\delta$  r)

theorem uniform_step_forces_all (L : Ladder) (h $\delta$  :  $\forall r$ , 1  $\leq$  L. $\delta$  r) :
   $\forall r$ , L.Decided r

```

```

| 0 => L.base
| r + 1 => L.mono (r + 1) (r + L.6 r) (L.step r
  (uniform_step_forces_all L h6 r))
  (Nat.add_le_add_left (h6 r) r)

/-! ## IX · freedom -/

def kineticRecord (_w : Bool) : Nat := 0

/-- THE RECORD WALL, the Bool model of the per-part bit: no reading
  of the kinetic record returns the bit;
  the wall on frames is record_wall_on_frames. -/
theorem record_wall :  $\neg \exists g : \text{Nat} \rightarrow \text{Bool}, \forall w, g (\text{kineticRecord } w) = w$  :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

/-- THE FIBRE IS TWO, the Bool model of the per-part bit: both
  values of the bit share one kinetic record;
  the wall on frames is record_wall_on_frames. -/
theorem fibre_is_two : ([false, true].filter (fun w => kineticRecord
  w == kineticRecord true)).length = 2 := by
  decide

/-- THE RECORD WALL ON FRAMES: no reading of a record that agrees on
  the calm and the gapless frame
  decides the value on every frame. -/
theorem record_wall_on_frames {α : Type} (rec : Frame → α) (h : rec
  calm = rec gapless) :
   $\neg \exists g : \alpha \rightarrow \text{Prop}, \forall F : \text{Frame}, (g (\text{rec } F) \leftrightarrow \text{Value } F) :=$ 
  fun (_, hg) => gapless_fails ((hg gapless).mpr (h  $\triangleright$  (hg calm).mpr
    calm_value))

def mulFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
    (List.range (n + 1)).filterMap fun b => if a * b = n then some
      (a, b) else none

theorem prime_shape :
  mulFibre 7 = [(1, 7), (7, 1)]  $\wedge$  (mulFibre 7).all (fun p => p.1
    != p.2) = true  $\wedge$ 
  (mulFibre 7).length = ([false, true].filter (fun w =>
    kineticRecord w == kineticRecord true)).length := by
  decide

/-! ## X · the triaxial lock -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
  xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b =>
    [false, true].map fun c => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r
  != (false, false, false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x ==
    rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
  (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
    [false, true].all (fun t1 => [false, true].all (fun t2 =>
      solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun
    r3 =>
      r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
      [false, true].all (fun t1 => [false, true].all (fun t2 =>
        [false, true].all (fun t3 =>
          solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true :=
  by decide

/-! ## XI · the closure, whole -/

theorem ym_closure :
  ( $\forall P : \text{Prop}, \neg \forall F : \text{Frame}, (P \leftrightarrow \text{Value } F) \wedge$ 
  ( $\forall F : \text{Frame}, \text{Confined } F \leftrightarrow \text{Value } F) \wedge$ 
  ( $\forall A : \text{ActualTheories}, \text{Value } A.F) \wedge$ 
  ( $\forall F : \text{Frame}, \text{Value } F \leftrightarrow \text{ExistsAll } F \wedge \text{GapAll } F) \wedge$ 

```

```

  (ExistsAll gapless  $\wedge \neg \text{GapAll gapless}) \wedge$ 
  ( $\forall (L : \text{Lattice}) (d : L.F.D), L.\beta d \leq L.\beta_0 \rightarrow \text{Exists } L.F d \wedge$ 
  Gapped L.F d)  $\wedge$ 
  ( $\forall (M : \text{Matter}) (d : M.D), 0 < M.\text{lowest } d) \wedge$ 
  Keyed Value  $\wedge$ 
  ( $\forall (F : \text{Frame}) (d : F.D), F.\text{built } d = \text{none} \rightarrow \neg \text{Confined } F) \wedge$ 
  ( $\forall (F : \text{Frame}) (d : F.D), F.\text{lowest } d = 0 \rightarrow \neg \text{Confined } F) \wedge$ 
  (GapAll empty  $\wedge \neg \text{ExistsAll empty}) \wedge$ 
  ( $0 < \text{weakLattice}.\beta () - \text{weakLattice}.\beta_0 \wedge \neg \text{Value weakLattice.F})$ 
  :=
  (given_is_not_the_value, act_is_the_value, ym_from_existence,
  two_parts,
  existence_does_not_give_gap, strong_coupling_decided,
  no_actual_gapless, value_is_keyed,
  no_theory_refutes, massless_refutes,
  gap_without_existence_is_empty, strong_does_not_reach_continuum)

/-! ## The root, undeniable in deed -/

/-- A self-grounding root, the structure of Appendix R in the same
  words: a type of acts, inhabited, every
  one of which instances the root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- The constructed root, in the words of Appendix R: one existent
  on the one-point domain, whose actuation is one. -/
def  $\Delta E_0$  : Unit → Int := fun _ => 1
def  $RA_0$  : Prop :=  $\forall x : \text{Unit}, 0 < \Delta E_0 x$ 

/-- The constructed domain grounds its root: its one act is a deed,
  and the deed actuates. -/
def ra0 : SelfGrounding  $RA_0$  := (Unit, (), fun _ => show (0 : Int)
  < 1 by decide)

/-- THE CONSTRUCTED ROOT IS THE ROOT ON THE ONE-POINT DOMAIN:
  Appendix R's root is the row's root on one existent with
  actuation one. -/
theorem  $RA_0$ _is_the_constructed_root :  $RA_0 \leftrightarrow \text{Root Unit}$  (fun _ => 1) :=
  Iff.rfl

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding
  root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R :=
  G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof
  of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R)  $\leftrightarrow$  R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed
  in a frame where the value holds
  and in one where it fails, so it forces the value on no uniform
  reading of the frames. -/
theorem undeniable_root_forces_no_value {R : Prop} (G :
  SelfGrounding R) :
  R  $\wedge$  Value calm  $\wedge \neg$  Value gapless  $\wedge \neg$  ( $\forall F : \text{Frame}, R \leftrightarrow \text{Value } F$ ) :=
  (G.instances G.anAct, calm_value, gapless_fails, fun h =>
    gapless_fails (h gapless (G.instances G.anAct)))

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read
  uniformly on the frames it gives no value;
  read on the row, with the row's existents and the row's actuation,
  it is the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
  R  $\wedge \neg$  ( $\forall F : \text{Frame}, R \leftrightarrow \text{Value } F$ )  $\wedge$  ( $\forall F : \text{Frame}, \text{RootOnRow } F \leftrightarrow$ 
  Value F)  $\wedge$  (RootOnRow calm  $\wedge \neg$  RootOnRow empty  $\wedge \neg$  RootOnRow
  gapless) :=
  (G.instances G.anAct, fun h => gapless_fails (h gapless
    (G.instances G.anAct)), root_on_row_is_the_value,
  root_on_row_is_keyed)

/-- THE LOCK, one theorem on no axiom: the constructed root holds
  with no hypothesis; every self-grounding
  root holds by its deed; no given statement is the value on every
  frame; the root read uniformly on the frames
  gives no value; read on the row the root is the act, the act is the
  value exactly, and both are keyed; the

```

```

closing theorem gives the value on every actual frame; no reading of
  the kinetic record returns the bit; and
no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
  Root Unit (fun _ => 1)  $\wedge$ 
  R  $\wedge$ 
  ( $\forall$  P : Prop,  $\neg \forall$  F : Frame, (P  $\leftrightarrow$  Value F))  $\wedge$ 
   $\neg$  ( $\forall$  F : Frame, R  $\rightarrow$  Value F)  $\wedge$ 
  ( $\forall$  F : Frame, RootOnRow F  $\leftrightarrow$  Confined F)  $\wedge$ 
  ( $\forall$  F : Frame, Confined F  $\leftrightarrow$  Value F)  $\wedge$ 
  (RootOnRow calm  $\wedge$   $\neg$  RootOnRow empty  $\wedge$   $\neg$  RootOnRow gapless)  $\wedge$ 
  (Confined calm  $\wedge$   $\neg$  Confined empty  $\wedge$   $\neg$  Confined gapless)  $\wedge$ 
  ( $\forall$  A : ActualTheories, Value A.F)  $\wedge$ 
  ( $\neg \exists$  g : Nat  $\rightarrow$  Bool,  $\forall$  w, g (kineticRecord w) = w)  $\wedge$ 
  ( $\forall$  n : Nat, ( $\forall$  d, d < n  $\rightarrow$  Exists (staged n) d  $\wedge$  Gapped (staged
n) d)  $\wedge$   $\neg$  Value (staged n)) :=
(constructed_root_holds, G.instances G.anAct,
given_is_not_the_value, (root_read_on_row_is_keyed G).2.1,
root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
act_is_keyed, ym_from_existence, record_wall,
finite_record_never_forces)

/-- THE LOCK ON THE CONSTRUCTED ROOT: the constructed root of
  Appendix R holds by its own act and forces
no value; read on the row the root is the value exactly; and the
  closing theorem gives the value on every
actual frame. -/
theorem the_lock_on_the_root :
  RA0  $\wedge$   $\neg$  ( $\forall$  F : Frame, RA0  $\rightarrow$  Value F)  $\wedge$  ( $\forall$  F : Frame, RootOnRow F
 $\leftrightarrow$  Value F)  $\wedge$  ( $\forall$  A : ActualTheories, Value A.F) :=
(ra0.instances ra0.anAct, fun h => gapless_fails (h gapless
(ra0.instances ra0.anAct))), root_on_row_is_the_value,
ym_from_existence)

/-- FROM EXISTENCE ALONE, in the sense the title carries: the
  closing theorem consumes existence read on
the row and nothing else; that reading is available on exactly the
  frames where the value holds; and no
statement given without the reading is the value on every frame. One
theorem, on no axiom. -/
theorem from_existence_alone :
  ( $\forall$  A : ActualTheories, Value A.F)  $\wedge$ 
  ( $\forall$  F : Frame, Nonempty { A : ActualTheories // A.F = F }  $\leftrightarrow$  Value
F)  $\wedge$ 
  ( $\forall$  P : Prop,  $\neg \forall$  F : Frame, (P  $\leftrightarrow$  Value F)) :=
(ym_from_existence, supply_iff, given_is_not_the_value)

end YMClose

/-! ## Cones, pinned as printed -/
/-- info: 'YMClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.calm_value
/-- info: 'YMClose.empty_fails' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.empty_fails
/-- info: 'YMClose.gapless_fails' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.gapless_fails
/-- info: 'YMClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.root_given
/-- info: 'YMClose.root_conservative' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.root_conservative
/-- info: 'YMClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.arrow_given
/-- info: 'YMClose.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.freedom_given
/-- info: 'YMClose.given_is_not_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.given_is_not_the_value
/-- info: 'YMClose.arrow_forces_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.arrow_forces_nothing
/-- info: 'YMClose.given_in_all_worlds' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.given_in_all_worlds
/-- info: 'YMClose.act_is_the_value' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.act_is_the_value
/-- info: 'YMClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.act_is_keyed
/-- info: 'YMClose.ym_from_existence' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.ym_from_existence
/-- info: 'YMClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.supply_iff

```

```

/-- info: 'YMClose.every_theory_lands' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.every_theory_lands
/-- info: 'YMClose.gates_exclusive' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.gates_exclusive
/-- info: 'YMClose.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.nothing_escapes
/-- info: 'YMClose.no_theory_refutes' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.no_theory_refutes
/-- info: 'YMClose.massless_refutes' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.massless_refutes
/-- info: 'YMClose.two_parts' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.two_parts
/-- info: 'YMClose.existence_does_not_give_gap' does not depend on
any axioms -/
#guard_msgs in #print axioms YMClose.existence_does_not_give_gap
/-- info: 'YMClose.gap_without_existence_is_empty' does not depend
on any axioms -/
#guard_msgs in #print axioms YMClose.gap_without_existence_is_empty
/-- info: 'YMClose.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.value_is_keyed
/-- info: 'YMClose.arrow_is_keyless' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.arrow_is_keyless
/-- info: 'YMClose.no_keyless_statement_is_the_act' does not depend
on any axioms -/
#guard_msgs in #print axioms YMClose.no_keyless_statement_is_the_act
/-- info: 'YMClose.pulse_does_not_certify' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.pulse_does_not_certify
/-- info: 'YMClose.strong_coupling_decided' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.strong_coupling_decided
/-- info: 'YMClose.strong_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.strong_is_load_bearing
/-- info: 'YMClose.wilson_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.wilson_is_load_bearing
/-- info: 'YMClose.strong_does_not_reach_continuum' does not depend
on any axioms -/
#guard_msgs in #print axioms YMClose.strong_does_not_reach_continuum
/-- info: 'YMClose.no_actual_gapless' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.no_actual_gapless
/-- info: 'YMClose.record_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.record_is_load_bearing
/-- info: 'YMClose.radiates_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.radiates_is_load_bearing
/-- info: 'YMClose.finite_record_never_forces' does not depend on
any axioms -/
#guard_msgs in #print axioms YMClose.finite_record_never_forces
/-- info: 'YMClose.no_symmetry_lifts' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.no_symmetry_lifts
/-- info: 'YMClose.uniform_step_forces_all' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.uniform_step_forces_all
/-- info: 'YMClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.record_wall
/-- info: 'YMClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.fibre_is_two
/-- info: 'YMClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.prime_shape
/-- info: 'YMClose.two_axes_leave_two' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.two_axes_leave_two
/-- info: 'YMClose.three_axes_lock_one' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.three_axes_lock_one
/-- info: 'YMClose.ym_closure' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.ym_closure
/-- info: 'YMClose.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.denial_reenacts_root
/-- info: 'YMClose.external_proof_adds_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms YMClose.external_proof_adds_nothing
/-- info: 'YMClose.undeniable_root_forces_no_value' does not depend
on any axioms -/

```

```

#guard_msgs in #print axioms YMClose.undeniable_root_forces_no_value
/-- info: 'YMClose.root_read_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.root_read_on_row_is_keyed
/-- info: 'YMClose.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.the_lock
/-- info: 'YMClose.from_existence_alone' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.from_existence_alone
/-- info: 'YMClose.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.indicator_pos
/-- info: 'YMClose.root_satisfiable' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.root_satisfiable
/-- info: 'YMClose.constructed_root_holds' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.constructed_root_holds
/-- info: 'YMClose.root_on_row_is_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.root_on_row_is_the_act
/-- info: 'YMClose.root_on_row_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.root_on_row_is_the_value
/-- info: 'YMClose.root_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.root_on_row_is_keyed
/-- info: 'YMClose.act_is_the_weakest_premise' does not depend on
any axioms -/
#guard_msgs in #print axioms YMClose.act_is_the_weakest_premise
/-- info: 'YMClose.existence_as_given_in_all_worlds' does not depend
on any axioms -/
#guard_msgs in #print axioms
YMClose.existence_as_given_in_all_worlds
/-- info: 'YMClose.nothing_given_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.nothing_given_refutes
/-- info: 'YMClose.only_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms YMClose.only_refuter
/-- info: 'YMClose.premise_excludes_singlet' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.premise_excludes_singlet
/-- info: 'YMClose.nothing_given_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.nothing_given_forces
/-- info: 'YMClose.reading_is_the_actuation' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.reading_is_the_actuation
/-- info: 'YMClose.fourth_gate_fails' does not depend on any axioms
-/
#guard_msgs in #print axioms YMClose.fourth_gate_fails
/-- info: 'YMClose.no_keyless_predicate_is_the_act' does not depend
on any axioms -/
#guard_msgs in #print axioms YMClose.no_keyless_predicate_is_the_act
/-- info: 'YMClose.strong_coupling_value' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.strong_coupling_value
/-- info: 'YMClose.fields_do_not_entail_weak_gap' does not depend on
any axioms -/
#guard_msgs in #print axioms YMClose.fields_do_not_entail_weak_gap
/-- info: 'YMClose.matter_gives_gap_half' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.matter_gives_gap_half
/-- info: 'YMClose.every_finite_record_never_forces' does not depend
on any axioms -/
#guard_msgs in #print axioms
YMClose.every_finite_record_never_forces
/-- info: 'YMClose.record_wall_on_frames' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.record_wall_on_frames
/-- info: 'YMClose.RA0 is the constructed root' does not depend on
any axioms -/
#guard_msgs in #print axioms YMClose.RA0 is the constructed root
/-- info: 'YMClose.the_lock_on_the_root' does not depend on any
axioms -/
#guard_msgs in #print axioms YMClose.the_lock_on_the_root

```

APPENDIX R · THE ROOT ON NO AXIOM AND NO POSIT

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, TOE_Zero.

lean, is carried verbatim from the root paper (Islam 2026h). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, SelfGrounding, SelfVerifying, ΔE_0 , RA_0 and ra_0 , and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (SelfGrounding); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x, 0 < \Delta E_0(x)$ (RA_0). On that domain the Root Axiom is a theorem with no axiom and no hypothesis (root_on_the_constructed_domain). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (the_floor_is_universal). Every denial of the root is a deed that instances it (denial_reenacts_root); the root is held by its deed (seated_undeniable); it is self-verifying (denial_instantiates); and no level stands above it (no_level_above). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (the_master_seal); it states the three counts side by side and no map between them. The row kernel repeats this kernel's three declarations ΔE_0 , RA_0 and ra_0 verbatim, so the identity of the constructed root across the two files is a textual check; there the constructed root is proved to be the row's root on the one-point domain with actuation one (RA_0 is the constructed root) and is bound to the row in one theorem (the_lock_on_the_root). The body of this paper reads the root on its row; this appendix is the root itself. Its SHA-256 is

```

0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238

```

```

/-
TOE_Zero.lean · Theory of Theories of Everything (TOE of All
TOEs): Existence Proves Existence Only by Motion
The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
axiom declared, no sorry, no native_decide, and
every theorem rests on no axiom at all, not even propositional
extensionality, quotients or choice.
I · The self-grounding root. Acts occur and every act instances
the root. Then every denial is an act and
re-enacts the root, the root is held by the act itself, and no
external proof adds anything to it: the
Omega Boundary, proved for every root that grounds itself,
universally, by quantification.
II · The constructed root. On the constructed domain the Root
Axiom is a theorem.
III · The Return and the line. On the integer quaternions i·j·k =
-1 and each unit squares to -1; conjugation is

```

```

    an involution and its fixed set on the chart is exactly the
    scalar line.
IV · The twelve gates, recovered three ways inside the integers of
    the quaternions: the twenty-four Hurwitz units
    counted twelve up to sign; the twelve even permutations with
    class equation 1, 3, 4, 4; and the norm-two
    shell of twenty-four, split twelve and twelve.
V · The regress terminated. Above a self-grounding root no level
    grounds it further.
VI · The master seal, every result bound in one theorem.
-/
namespace TOE0

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the
    root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and
    re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
    the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
    root holds, every act re-enacts it, and no
    external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
  Prop, (Q → R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one
    existent actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed,
    and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
  < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
  ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b *
  y.a + x.c * y.d - x.d * y.c,
  x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b *
  y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)

```

```

def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i·j·k = -1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to -1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj =
  qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in -2 .. 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q =>
  q.conj.conj == q) = true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the
    chart. -/
theorem the_line_is_the_fixed_set :
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
  q.d == 0)) = true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all
    odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 !=
  0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates -2 .. 2 hold every unit (doubled norm four)
    and every element of the norm-two shell (doubled
    norm eight), since a doubled coordinate of absolute value three
    or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of
    each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then
  0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧
  (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
  8 && hurwitz a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split
    twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
  shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
  (shell.filter (fun p => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1,
  2, 3].flatMap fun c =>
  [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length
  == 4 then some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
  j)).countP

```

```

    fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p %
  2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
  => compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
| [] => [n]
| m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a
  class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
| 0, _ => []
| _ + 1, [] => []
| fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
  gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
    (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
  further ground of the root is idle; the root needs
  no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ => G.instances G.act

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
  on the constructed domain; the Return closes;
  the line is the fixed set; and the twelve gates are recovered
  three ways. -/
theorem the_master_seal :
  (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.act, R) ∧ (∀ Q :
    Prop, (Q → R) → R)) ∧
  RA₀ ∧ (q₁.mul qj).mul qk = qneg ∧
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true ∧
  (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
  (shell.filter upToSign).length = 12 ∧
  collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
  = [1, 3, 4, 4] :=
  (the_floor_is_universal, root_on_the_constructed_domain,
  the_return, the_line_is_the_fixed_set,
  twenty_four_units_twelve_gates.2, the_class_equation.1,
  the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms
  -/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms
  -/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on
  any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return

```

```

/-- info: 'TOE0.units_square_to_minus_one' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on
  any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend
  on any axioms -/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

```

The root's axes are proved in the second kernel, R.2, Triaxial_Actuation.lean (32 theorems, every one on no axiom; SHA-256 566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (actuation_is_triaxial, registration_is_forced, the_only_special_cut, tongue_freedom, every_root_actuates, the_root_carries_three_axes). The root's grade is read in the third kernel, R.3, Root_Grade_Ledger.lean (18 theorems, every one on no axiom; SHA-256 c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8). On the grade ledger of the Master Codex (Islam 2026k), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inherits the root's grade exactly (a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026j), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the paper that reads them on its row cites them for exactly what they state.

```

/-
Triaxial_Actuation.lean · actuation is triaxial

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

An actuation is a self-grounding root whose acts carry a
direction: each act has a side and a height,
and reversing the deed reverses the side and keeps the height.
From the direction alone the kernel
constructs three axes and proves them forced:
I The fold: reversing the deed is an involution.
II The registration: it keeps the height and forgets the side;
it is blind to the fold, closed with
it under combination, and it is the only height-keeping map
onto the seat.
III The seat: the fold's fixed set, where the fold and the
registration both fix the act, and off
which neither does.
IV The third axis: off the seat the fold's orbit has two acts
with one record, so the record leaves
exactly the orientation open; every reading blind to the fold
reads both alike.
V The root through the axes: a reversed deed is still an act
and instances the root; the axes are
carried by every actuation, and they force no orientation.
VI The bridge atom: the integers, with negation as the fold, are
an actuation.
VII The triaxial seal.
VIII The Tongue: no reading of the record returns the orientation;
the seat is the only special cut.
IX Every root actuates: a self-grounding root is the root of an
actuation over its own acts, so the
root carries the three axes; the registration is the fold's
record and lands on the seat, where
the orientation an act and its reversal share stays open; the
even pair leaves two acts, the
orientation locks one; and a degenerate direction, one side
only, is the characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction
-/

/-- The direction of a deed: a side with a reversal, whose one fixed
side is the zero side. -/
structure Direction where
  Side      : Type
  rev       : Side → Side
  zero      : Side
  rev_rev   : ∀ s, rev (rev s) = s
  rev_zero  : rev zero = zero
  fix_zero  : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a
side and a height. -/
structure Actuation (R : Prop) where
  dir      : Direction
  Height   : Type
  anAct    : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

/-- The fold: reversing the deed reverses the side and keeps the
height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side
× A.Height := (A.dir.rev p.1, p.2)

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
A.Height := (A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop :=
fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side ×
A.Height) : fold A (fold A p) = p := by
cases p with
| mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw
[A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have
one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : reg A (fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side ×
A.Height) : (reg A p).2 = p.2 := rfl

/-- The even pair is closed under combination: folding a record
changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) :
fold A (reg A p) = reg A p := by
show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw
[A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height)
: reg A (reg A p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the
height and lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side ×
A.Height → A.dir.Side × A.Height)
(keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p,
g p = reg A p := by
intro p
have hs : (g p).1 = A.dir.zero := by
have hf := lands p
have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
exact A.dir.fix_zero _ h1
have hh := keeps p
cases hg : g p with
| mk s h =>
rw [hg] at hs hh
show (s, h) = (A.dir.zero, p.2)
rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : OnSeat A p ↔ p.1 = A.dir.zero :=
(fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
fun h => by
cases p with
| mk s t =>
show (A.dir.rev s, t) = (s, t)
have hs : s = A.dir.zero := h
rw [hs, A.dir.rev_zero])

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the
registration both fix the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : OnSeat A p) :
fold A p = p ∧ reg A p = p := by
refine (h, ?_)
have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
cases p with
| mk s t =>
have hs : s = A.dir.zero := hz
show (A.dir.zero, t) = (s, t)

```

```

    rw [hs]

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration
fixes the act. -/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height)
(h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A p ≠ p := by
  refine (h, fun hr => h ?_)
have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act
and its reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p :
  A.dir.Side × A.Height) (h : ¬ OnSeat A p) :
  fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(h, rfl)

/-- Every reading blind to the fold reads an act and its reversal
alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f :
  A.dir.Side × A.Height → α)
  (blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) :
  f (fold A p) = f p :=
blind p

/-- The registration is such a reading, and so is every reading of
the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α :
  Type} (k : A.dir.Side × A.Height → α)
  (p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p)
:= rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the
denial spends the deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side ×
  A.Height) : R :=
A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.anAct

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations
are acts, both instance the root, and
the record reads them alike; the triaxial structure leaves the one
orientation open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side
  × A.Height) (h : ¬ OnSeat A p) :
  R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, -(s) = s
| Int.ofNat 0 => rfl
| Int.ofNat (n + 1) => rfl
| Int.negSucc n => rfl

theorem self_neg_zero : ∀ v : Int, -v = v → v = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (n + 1), h => Int.noConfusion h
| Int.negSucc n, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := (Int, fun s => -s, 0, int_neg_neg, rfl,
  self_neg_zero)

/-- The bridge atom: points (d, t), the fold (d, t) ↔ (-d, t), the
registration (d, t) → (0, t). -/
def atom : Actuation True := (intDir, Int, ((0 : Int), (0 : Int)),
  fun _ => True.intro)

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly
when d = 0, the registration
leaves it unchanged exactly then, and off the seat the record leaves
the orientation. -/
theorem the_bridge_atom (d t : Int) :
  (OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0)
  ∧

```

```

  (d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t))
    = reg atom (d, t)) :=
(seat_iff_zero_side atom (d, t),
  (fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl),
  fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
    (seat_iff_zero_side atom (d, t)).mp hs)))

/-- The atom is not characterless: off the seat it has two acts with
one record. -/
theorem the_atom_is_not_characterless :
  fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
  atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
    : Int)) :=
(fun h => Int.noConfusion (congrArg Prod.fst h), rfl)

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an
involution, a registration that
keeps the height, forgets the side and is the only such map onto the
seat, and a seat where both fix
the act and off which neither does; off the seat the record leaves
exactly the orientation; every act
and its reversal instance the root; and the axes force no
orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
  (∀ p, fold A (fold A p) = p) ∧
  (∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A
    (reg A p) = reg A p) ∧
  (∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
    (∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p) → ∀ p, g p = reg
      A p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
    (∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) =
      reg A p) :=
(fold_involutive A, fun p => (reg_forgets_side A p,
  reg_keeps_height A p, fold_of_reg A p),
  registration_is_forced A, seat_cancels_both A, off_seat_both A,
  axes_force_no_orientation A)

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the
bridge atom is an actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
  (∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
    (∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
    (∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A
      p)) ∧
  (∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
  (fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
    atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
      : Int))) :=
(fun _ A => (fold_involutive A, seat_cancels_both A,
  fun p h => record_leaves_the_orientation A p h),
  fun d t => seat_iff_zero_side atom (d, t),
  the_atom_is_not_characterless)

/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the
orientation. If a record is blind to the fold
and an orientation is flipped by it, the orientation is not a
function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (p : α → β) (g : β →
  Bool) (d : α → Bool) (x : α)
  (blind : p (τ x) = p x) (flip : d (τ x) = !d x) : d ≠ fun y => g
  (p y) := by
  intro h
  have h1 : d (τ x) = g (p (τ x)) := congrFun h (τ x)
  have h2 : d x = g (p x) := congrFun h x
  rw [blind] at h1
  rw [h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots
cancel; nowhere else do the fold
and the registration both fix an act. -/
theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side ×
  A.Height) :

```

```

(fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
(fun h => h.1, fun h => seat_cancels_both A p h)

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its
reversal alike and keeps the height, and two
acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q :
A.dir.Side × A.Height) :
reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
(rfl, (fun h => show (reg A p).2 = (reg A q).2 from congrArg
Prod.snd h, fun h => by
cases p with
| mk s t =>
cases q with
| mk s' t' =>
have ht : t = t' := h
show (A.dir.zero, t) = (A.dir.zero, t')
rw [ht]))

/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat
point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side ×
A.Height) : OnSeat A (reg A p) :=
fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and
its reversal land on one seat point, the record
of both; at that place the record has left exactly the orientation
open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : ¬ OnSeat A p) :
OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
(reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the
fold, the record among them, reads an act and
its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : ¬ OnSeat A p) {α : Type}
(f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f
q) :
fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
(h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation,
the side, determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q :
A.dir.Side × A.Height)
(hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
cases p with
| mk s t =>
cases q with
| mk s' t' =>
have ht : t = t' := congrArg Prod.snd hr
have hs' : s = s' := hs
rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is
on the seat, and the fold and the
registration are both the identity: one scalar, no fold, no seat to
lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h :
A.dir.Degenerate) (p : A.dir.Side × A.Height) :
OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
(hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
have h0 : (1 : Int) = 0 := h (1 : Int)
absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the
actuation over its own acts, with the integer
direction; the root's deeds sit at the zero side and instance the
root as before. -/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=
(intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2)

```

```

theorem every_root_actuates (S : SelfGrounding R) :
(∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧
(∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
¬ S.toActuation.dir.Degenerate :=
(fun _ => rfl,
fun a h =>
have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation
(1 : Int), a).mp h
absurd h0 (by decide),
intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its
actuation carries the fold, an involution; the
registration, keeping the height, forgetting the side, the only
height-keeping map onto the seat; and the seat,
where both fix the act; and it is not characterless: at every height
an act off the seat, its reversal a second
act with the same record, both instantiating the root, the orientation
between them forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
(∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
(∀ p, reg S.toActuation (fold S.toActuation p) = reg
S.toActuation p ∧ (reg S.toActuation p).2 = p.2) ∧
(∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p,
OnSeat S.toActuation (g p)) →
∀ p, g p = reg S.toActuation p) ∧
(∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg
S.toActuation p = p) ∧
(∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg
S.toActuation ((1 : Int), a)) ∧
(∀ p, ¬ OnSeat S.toActuation p →
R ∧ R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold
S.toActuation p) = reg S.toActuation p) :=
(fold_involutive S.toActuation,
fun p => (reg_forgets_side S.toActuation p, reg_keeps_height
S.toActuation p),
registration_is_forced S.toActuation,
seat_cancels_both S.toActuation,
fun a => record_leaves_the_orientation S.toActuation ((1 : Int),
a) ((every_root_actuates S).2.1 a),
axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms
-/-
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms
-/-
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds
/-- info: 'TriAct.axes_force_no_orientation' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation

```

```

/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_triaxial' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_triaxial
/-- info: 'TriAct.the_triaxial_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.the_triaxial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes

/-
Root_Grade_Ledger.lean · the root at theorem grade, and RA-Tongue
co-location with it

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

The floor Codex (Master Codex v5.1.0, Codex.lean) grades every
claim on one ledger and joins claims
at the weakest link. It declares the root as `axiom RA` and pins
`raClaim.grade = .premise`; its
co-location claim, the root joined with the wall, therefore
inherits `.premise`. This kernel carries
the same ledger and the same join law with no axiom declared,
warrants the root by a theorem with no
axiom and no hypothesis, and proves that the co-location claim
then stands at theorem grade.
I The grade ledger and the weakest-link join, as in the Codex.
II The root, a theorem: the constructed domain of TOE_Zero.lean,
warranted on no axiom.
III The wall, a theorem: the Codex's K4 frame, its target not a
function of its record, proved on
no axiom through Tongue freedom.
IV The claims: the root at theorem grade, the wall at theorem
grade, and their co-location.
V Why the Codex read premise: a join's grade is its weakest
link, and co-location's grade is the
root's grade exactly; with the root at theorem grade,
co-location is a theorem.
VI The ledger seal.
VII The grade read from the warrant, as the Codex reads it: a
root with a warrant, a proof of its
statement, is theorem grade; a root posited without one, the
Codex's `axiom RA`, is premise grade;
the two claims of section IV carry exactly the grades this
rule reads.

```

```

-/

namespace RGL

/-- ## I · the grade ledger, as in the Codex -/

inductive Grade : Type
| premise | corroboration | operational | structural | engineering
| conditional | theoremConditional | analytic | theorem
deriving DecidableEq

def Grade.rank : Grade → Nat
| .premise => 0 | .corroboration => 1 | .operational => 2 |
.structural => 3
| .engineering => 4 | .conditional => 5 | .theoremConditional => 6
| .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a
else b

/-- A claim carries its statement, its grade and its warrant, a
proof of the statement. -/
structure Claim : Type where
stmt : Prop
grade : Grade
warrant : stmt

/-- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
stmt := c₁.stmt ∧ c₂.stmt
grade := Grade.weakest c₁.grade c₂.grade
warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade =
Grade.weakest c₁.grade c₂.grade := rfl

/-- ## II · the root, a theorem (the constructed domain of
TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
Act : Type
anAct : Act
instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
G.instances G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no
hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-- ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an
orientation is flipped by it, the
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (p : α → β) (g : β →
Bool) (d : α → Bool) (x : α)
(blind : p (τ x) = p x) (flip : d (τ x) = !d x) : d ≠ fun y => g
(p y) := by
intro h
have h1 : d (τ x) = g (p (τ x)) := congrFun h (τ x)
have h2 : d x = g (p x) := congrFun h x
rw [blind] at h1
rw [h2] at h1
rw [h1] at flip
cases hd : d x with
| false => rw [hd] at flip; exact Bool.noConfusion flip
| true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
| 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437
| 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
| _ => 0

def tau : Nat → Nat := fun i =>

```

```

    if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
  (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i % 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero
  target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom
and no hypothesis. -/
def raClaim₀ : Claim where
  stmt := RA₀
  grade := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade := .theorem
  warrant := K4.wall

/-- RA-Tongue co-location: the root joined with the wall, as in the
Codex. -/
def colocClaim₀ : Claim := Claim.join raClaim₀ wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison:
the same statement form, at
premise grade. -/
def raClaimPremise : Claim where
  stmt := RA₀
  grade := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's
grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem
= g := by
  cases g <|> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a
theorem, so the join reads the
root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
  (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the
co-location claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join
  raClaimPremise wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim₀.grade = .theorem := rfl

/-- RA-TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim₀.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall,
both proved. -/
theorem coloc_holds : colocClaim₀.stmt := colocClaim₀.warrant

/-! ## VI · the ledger seal -/

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no
hypothesis; the wall is a theorem; their

```

```

co-location holds and stands at theorem grade; and co-location's
grade is the root's grade exactly,
which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
  RA₀ ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
  colocClaim₀.stmt ∧ colocClaim₀.grade = .theorem ∧
  (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
  (Claim.join raClaimPremise wallClaim).grade = .premise :=
  (root_on_the_constructed_domain, K4.wall, coloc_holds,
  coloc_is_theorem_grade, premise_root_gives_premise_coloc)

/-! ## VII · the grade read from the warrant, as the Codex reads it
-/

/-- The evidence a ledger entry carries for a statement: posited
with no warrant, or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
| posited
| warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade
is a proof with its axioms printed. A root
that comes with a warrant, a proof of its statement, is theorem
grade; a root posited with none, as the Codex
declares 'axiom RA', is premise grade. -/
def rootGrade : Evidence RA₀ → Grade
| .warranted _ => .theorem
| .posited => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II
carries its warrant, so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
  rootGrade (.warranted root_on_the_constructed_domain) = .theorem
:= rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule
reads premise, the Codex's own reading of
its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited =
.premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV
carries the grade the rule reads from its warrant,
and the comparison claim carries the grade the rule reads from none.
-/
theorem the_ledger_reads_the_warrant :
  raClaim₀.grade = rootGrade (.warranted raClaim₀.warrant) ∧
  raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant
and premise from none, never the other way;
whatever label an entry carries, the warranted root's grade under
the rule is theorem. -/
theorem the_grade_is_not_elected :
  (∀ w : RA₀, rootGrade (.warranted w) = .theorem) ∧ rootGrade
  .posited ≠ .theorem ∧
  (∀ w : RA₀, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h =>
  Grade.noConfusion h)

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-- info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-- info: 'RGL.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-- info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-- info: 'RGL.K4.register_even_at_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-- info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-- info: 'RGL.K4.wall' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.K4.wall
/-- info: 'RGL.weakest_with_theorem' does not depend on any axioms
-/

```

```

#guard_msgs in #print axioms RGL.weakest_with_theorem
/-- info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-- info: 'RGL.premise_root_gives_premise_coloc' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend
on any axioms -/
#guard_msgs in #print axioms
RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected

```

AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full in the programme's master reference (Islam 2026i).

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. One claim was held at the model's insistence: the matter face is graded at its weakest field, the premise on the massless case, beside the record of confinement, and read on the pure-gauge sector only, because the light pion of chiral QCD is colour-neutral and a massless colour singlet would refute the premise.

The register. In the programme's vocabulary: the form, the act's identity with the value, the universal closure, the division and the deduction of the lattice proved part stand at $[\{\epsilon\}, T]$ on no axiom, and the conclusion of the lattice proved part at the grade of its two cited theorems, the same rule as the matter face; the matter face at the grade of its weakest field, premise grade; the triaxial reading at $[\{\epsilon\}, S]$; the continuum value at premise grade on the act; $\Delta M = 0$ on the cited physics. The act is the row's least-erasure posit read as existence confined; the row carries no defeater. The freedom cut, prime-as-freedom and the triaxial lock are applied on the row, with the lessons of the Riemann, P versus NP, Navier--Stokes and Hodge closures combined, and the keyless and keyed denial of the programme's bridge and operating system.

Audit. This edition has not yet been prosecuted by the

external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapient/Trisduction.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (*every_root_actuates, degenerate_is_characterless*). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat	every_root_actuates, the_root_carries_three_axes, actuation_is_tri axial, registration_is_forced (Appendix R.2)
the registration is the only height-keeping map onto the seat	registration_is_forced (R.2)
the axes force no orientation	axes_force_no_orientation (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	fold, denial_instances_root (R.2); denial_reenacts_root, denial_instantiates (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination	registration_is_the_folds_record, reg_forgets_side, reg_keeps_height, fold_involution, fold_of_reg (R.2)
the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	readings_of_the_record_are_blind, tongue_freedom (R.2); symmetry_is_keyless, line_property_is_keyed, exactly_one_stands (Islam 2026k, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)
the seat is the fixed set, where both blind spots cancel; off the seat both stand	seat_iff_zero_side, seat_cancels_both, off_seat_both, the_only_special_cut (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	reg_lands_on_seat, open_at_the_seat, record_leaves_the_orientation, readings_of_the_record_are_blind (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	degenerate_is_characterless, intDir_not_degenerate, every_root_actuates (R.2)

Clause of the statement	Receipt
the bridge atom is an actuation, and not characterless	the_bridge_atom, the_atom_is_not_characterless, the_tri axial_seal (R.2)
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	the_return, the_line_is_the_fixed_set, twenty_four_units_twelve_gates, the_class_equation, the_master_seal (R.1)
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it	a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal (R.3)
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	even_pair_leaves_two, orientation_locks_the_act, seat_cancels_both (R.2); in its own coordinates, for independent axes in $\mathbb{IF}_2^3$, two_axes_leave_two, three_axes_lock_one (this paper's kernel), a second statement
the root alone forces no value; the root read on the row is the value	undeniable_root_forces_no_value; root_on_row_is_the_value, act_is_the_value (this paper's kernel)
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	every_root_actuates, the_root_carries_three_axes (R.2), stated for every self-grounding root and read at the constructed root ra_0 of this paper's kernel, whose structure repeats R.2's SelfGrounding field for field; this kernel states no seat of its own; its fold on the value bit is the swap of the two worlds that share one record, the Bool model (record_wall, fibre_is_two), and the seat its actuation carries is the record both deeds land on; the row ledger of Islam (2026d, Book III), cited

Clause of the statement	Receipt
-------------------------	---------

REFERENCES

- Creutz, M. 1980. Monte Carlo study of quantized SU(2) gauge theory. *Physical Review D* 21: 2308--2315.
- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE* 28, 625--635.
- Gross, D. J. and F. Wilczek. 1973. Ultraviolet behavior of non-abelian gauge theories. *Physical Review Letters* 30: 1343--1346.
- Islam, M. F. 2026a. A formal completed proof of the Navier--Stokes closure from existence alone. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026b. The veil and the wall. Zenodo. doi:10.5281/zenodo.21389758.
- Islam, M. F. 2026c. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026d. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026e. What exists is realized: the completed formal closure of the Hodge question. Zenodo. doi:10.5281/zenodo.22701510.
- Islam, M. F. 2026f. PhysOS^T: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.
- Islam, M. F. 2026g. The proton is the lock: three colors, one singlet, and the gap made flesh. Zenodo. doi:10.5281/zenodo.22986559.
- Islam, M. F. 2026h. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.
- Islam, M. F. 2026i. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.
- Islam, M. F. 2026j. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.
- Islam, M. F. 2026k. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapients/Trisduction, GitHub.
- Jaffe, A. and E. Witten. 2006. Quantum Yang--Mills theory. In *The Millennium Prize Problems*, 129--152. Clay Mathematics Institute.
- Morningstar, C. J. and M. Peardon. 1999. Glueball spectrum from an anisotropic lattice study. *Physical Review D* 60: 034509.
- Navas, S. et al. (Particle Data Group). 2024. Review of particle physics. *Physical Review D* 110: 030001.
- Osterwalder, K. and E. Seiler. 1978. Gauge field theories on a lattice. *Annals of Physics* 110: 440--471.

Politzer, H. D. 1973. Reliable perturbative results for strong interactions? *Physical Review Letters* 30: 1346--1349.

Seiler, E. 1982. *Gauge Theories as a Problem of Constructive Quantum Field Theory and Statistical Mechanics*. Lecture Notes in Physics 159. Berlin: Springer.

Wilson, K. G. 1974. Confinement of quarks. *Physical Review D* 10: 2445--2459.

End of manuscript